

▼ Đề cương chi tiết ✎

Collapse all ⋮

whiteboard link : <https://excalidraw.com/#room=8782ef0c1b762aedd9fd,2qfvyXB7GwQy4H8il4aqhg>

ĐỀ CƯƠNG CHI TIẾT HỌC PHẦN

1. Tên học phần:

Tiếng Việt: Lập trình .NET 1 + BTL

Tiếng Anh: Windows Forms Programming in .NET

Mã số học phần: 7080108

Số tín chỉ học phần: 3 (2-2-5)

Số tiết học phần:

Lý thuyết: 30	Bài tập lớn: 15	Thực hành: 0
Thực tập: 0	Đồ án: 0	Tự học: 75

2. Đơn vị quản lý học phần

2.1. Giảng viên giảng dạy:

Mã giảng viên

2.2. Bộ môn: Công nghệ Phần mềm

2.3. Khoa: Công nghệ Thông tin

3. Điều kiện học học phần (7080108)

4. Môn học tiên quyết: Cơ sở dữ liệu (7080207), Kỹ thuật lập trình hướng đối tượng với C++ +BTL (7080216)

?

Giúp sinh viên lập trình Windowform sử dụng nền tảng công nghệ .NET.

4.1. Về kiến thức

- 4.1.1 Kiến thức về .Net Core
- 4.1.2 Kiến thức về giao diện người dùng (User Interface)
- 4.1.3 Các kiểu dữ liệu & Bẫy lỗi trong C#
- 4.1.4 Lập Trình Hướng Đối Tượng với C#
- 4.1.5 Cơ sở dữ liệu & C#

4.2. Kỹ năng

- 4.2.1. Sử dụng được phần mềm Visual Studio .Net để xây dựng được ứng dụng window form (từ đơn giản đến phức tạp)
- 4.2.2. Sử dụng phần mềm Microsoft SQL Server để xây dựng cơ sở dữ liệu cho ứng dụng cần phát triển trên C#
- 4.2.3. Rèn luyện kỹ năng làm việc theo nhóm thông qua làm đề tài bài tập lớn, thuyết trình và thảo luận

4.3. Chuẩn đầu ra học phần

Sau khi hoàn thành học phần này, sinh viên có thể:

1. Nắm được những khái niệm, phương pháp, thuật toán, kỹ thuật và các ứng dụng của lập trình Net1;
2. Nắm được các kiểu dữ liệu trên nền tảng .Net;
3. Thiết kế giao diện trên nền tảng .Net;
4. Lập trình với CSDL trên nền tảng .Net.

Các chuẩn đầu ra học phần trên phù hợp với các chuẩn đầu ra chương trình đào tạo ngành Công nghệ thông tin như sau:

Select all

0 selected

-	-	-	√	√	√		√		√	√	√	√	√	√	√	-	-
-	-	-	√	√	√		√		√	√	√	√	√	√	√		
-	-	-	√	√	√		√		√	√	√	√	√	√	√		
-	-	-	√	√	√		√		√	√	√	√	√	√	√		
-	-		√	√	√		√		√	√	√	√	√	√	√		

Mô tả tóm tắt nội dung học phần

Nội dung học phần gồm 5 chương:

- Chương 1: Giới thiệu về .Net Core
- Chương 2: Thiết kế giao diện người dùng
- Chương 3: Kiểu dữ liệu và bắt lỗi trong C#
- Chương 4: Lập trình hướng đối tượng với C#
- Chương 5: Lập trình CSDL với C#

7. Cấu trúc nội dung học phần

Bảng 1. Nội dung học phần

6.1. Lý thuyết			
Đề mục	Nội dung	Số tiết	CDR HP

Select all

0 selected

1.1	Giới thiệu về .NET & C#	1	
1.2	Ngôn ngữ lập trình C#	2	
1.3	Thư viện lớp cơ sở .NET	2	
Chương 2	Giao diện người dùng (User Interface)	5	CĐR 3
2.1	Lập trình sự kiện (Event Programming)	1	
2.2	Cửa sổ (Biểu mẫu, Form)	1	
2.3	Các điều khiển cơ bản trong C#	1	
2.4	Các điều khiển nâng cao	1	
2.5	Thực đơn (Menu)	1	
Chương 3	Các kiểu dữ liệu & Bẫy lỗi trong C#	5	CĐR 2, 4
3.1	Sử dụng các kiểu dữ liệu	1	
3.2	Kiểm liệt kê, mảng & tập hợp	2	
3.3	Xử lý lỗi trong C#	2	
Chương 4	Lập Trình Hướng Đối Tượng với C#	5	CĐR 4
4.1	Thuộc tính (Properties)	1	

Select all

0 selected

4.4	Giao diện (Interface)	2	
Chương 5	Cơ sở dữ liệu & C#	10	CĐR 4
5.1	Một số khái niệm về cơ sở dữ liệu	2	
5.2	Ngôn ngữ hỏi SQL	2	
5.3	Giới thiệu về ADO.NET	6	
6.2. Thực hành			
Đề mục	Nội dung	Số tiết	CĐR HP
Chương 1	Giới thiệu về .Net Core	6	CĐR 1, 2
Bài 1	Làm quen với Editor MS Visual Studio	2	
Bài 2	Thực hành sử dụng biến , hằng, toán tử.	4	
Chương 2	Giao diện người dùng (User Interface)	6	CĐR 3
Bài 3	Ứng dụng Window form đầu tiên	2	
Bài 4	Xử lý chuỗi	4	

Select all

0 selected

Bài 5	Thực hành các cấu trúc điều khiển	2	
Bài 6	Thiết kế MDI form và Menu	4	
Chương 4	Lập Trình Hướng Đối Tượng với C#	6	CĐR 4
Bài 7	Thực hành viết Lớp (Class) trong C#	2	
Bài 8	Thực hành hướng đối tượng trong C#	2	
Bài 9	Sử dụng các đối tượng danh sách trong lập trình Window form	2	
Chương 5	Cơ sở dữ liệu & C#	6	CĐR 4
Bài 10	Các thao tác cơ bản trên SQL Server	2	
Bài 11	Đọc dữ liệu từ SQL Server lên form	2	
Bài 12	Làm query với các thao tác trên Database	1	
Bài 13	Làm báo cáo với Crystal Report	1	

8. Phương pháp giảng dạy

- Giảng dạy lý thuyết kết hợp bài tập
- Tổ chức thảo luận, thuyết trình, hướng dẫn làm bài tập lớn theo nhóm

9. Nhiệm vụ của sinh viên

Sinh viên phải thực hiện các nhiệm vụ sau:

- Chủ động chuẩn bị các nội dung và thực hiện giờ tự học theo mục 11.

10. Đánh giá kết quả học tập của sinh viên

10.1. Cách đánh giá

Sinh viên được đánh giá tích lũy học phần như sau:

Bảng 2. Đánh giá học phần

TT	Điểm thành phần	Quy định	Trọng số
1	Điểm chuyên cần	Số tiết tham dự học/tổng số tiết	10%
2	Điểm bài tập	Số bài tập đã làm/số bài tập được giao	
3	Điểm bài tập nhóm	- Báo cáo, thuyết minh trên lớp - Được nhóm xác nhận có tham gia	30%
4	Điểm kiểm tra giữa kỳ	- Thi trên máy tính	
5	Điểm thi kết thúc học phần	- Thi trên máy tính, thi vấn đáp - Tham dự đủ 80% tiết lý thuyết và 100% giờ thực hành	60%

10.2. Cách tính điểm

- Điểm đánh giá thành phần và điểm thi kết thúc học phần được chấm theo thang điểm 10 (từ 0 đến 10), làm tròn đến một chữ số thập phân.
- Điểm học phần là tổng điểm của tất cả các điểm đánh giá thành phần của học phần nhân với trọng số tương ứng. Điểm học phần tính theo thang điểm 10 làm tròn đến một chữ số thập phân, sau đó được quy đổi sang điểm chữ và điểm số theo thang điểm 4 theo quy định về công tác đào tạo của Nhà trường.

Select all

0 selected

		thường xuyên			
1		X	X	X	X
2		X	X	X	X
3	X	X	X	X	X
4	X	X	X	X	X
5	X	X	X	X	X
6	X	X	X	X	X
7		X	X	X	X
8		X	X	X	X

11. Tài liệu học tập

- [1] Phương Lan, Lập trình Windows với C#.net, Lao động - Xã hội, 2002
- [2] Phương Lan, Phạm Hữu Khang, Hoàng Đức Hải, Kỹ Thuật Lập trình ứng dụng C#.net toàn tập, Lao động xã hội, 2005
- [3] Đinh Xuân Lâm, Những bài thực hành Visua Basic.net căn bản, Đại học Mở - Địa Chất, 2004
- [4] Phạm Hữu Khang, Ví dụ & bài tập Visual Basic.NET.Lập trình hướng đối tượng

Select all

0 selected

12. Hướng dẫn tự học của học phần

Bảng 3. Nội dung chuẩn bị

Tuần	Nội dung	Lý thuyết (tiết)	Bài tập (tiết)	Thực hành (tiết)	Sinh viên cần chuẩn bị
1	Chương 1: Giới thiệu về .Net Core 1.1. Giới thiệu về .NET & C# 1.2. 1.2 Ngôn ngữ lập trình C# 1.3. Thư viện lớp cơ sở .NET	10	3		+ Nghiên cứu nội dung chương 1 của tài liệu tham khảo [1]. + Lập danh sách nhóm làm việc, đăng ký đề xuất đề tài bài tập lớn, tham khảo danh sách đề tài bài tập lớn đã nhận
2	Chương 2: Giao diện người dùng (User Interface) 2.1. Lập trình sự kiện (Event Programming) 2.2. Cửa sổ (Biểu mẫu, Form) 2.3. Các điều khiển trong C# 2.4. Các điều khiển khác 2.5. Menu	10	3		+ Nghiên cứu nội dung chương 2 của tài liệu tham khảo [1]. + Làm bài tập liên quan trong tài liệu tham khảo [2], [3], [4] + Làm bài tập lớn theo nhóm

Select all

0 selected

3	3.1. Kiểu liệt kê, mảng & tập hợp 3.2. Kiểu liệt kê, mảng & tập hợp 3.3. Xử lý lỗi trong C#				quan trọng tài liệu tham khảo [2], [3], [4] + Làm bài tập lớn theo nhóm
4	Chương 4: Lập Trình Hướng Đối Tượng với C# 4.1.Thuộc tính (Properties) 4.2.Đa hình (Polymorphism) 4.3.Thừa kế (Inheritance) 4.4.Giao diện (Interface)	10	3		+ Nghiên cứu nội dung chương 4 của tài liệu tham khảo [1]. + Làm bài tập liên quan trọng tài liệu tham khảo [2], [3], [4] + Làm bài tập lớn theo nhóm
5	Chương 5: Cơ sở dữ liệu & C# 5.1. Một số khái niệm về cơ sở dữ liệu 3.2. Ngôn ngữ hỏi SQL 3.3. Giới thiệu về ADO.NET	20	3		+ Nghiên cứu nội dung chương 5 của tài liệu tham khảo [1]. + Làm bài tập liên quan trọng tài liệu tham khảo [2], [3], [4] + Làm bài tập lớn theo nhóm

Select all

0 selected

+ Add an activity or resource



Test1



Chương 1 Giới thiệu về .Net Core



Chương 1: Giới thiệu về .NET Core

Số tiết dự kiến: 5 tiết

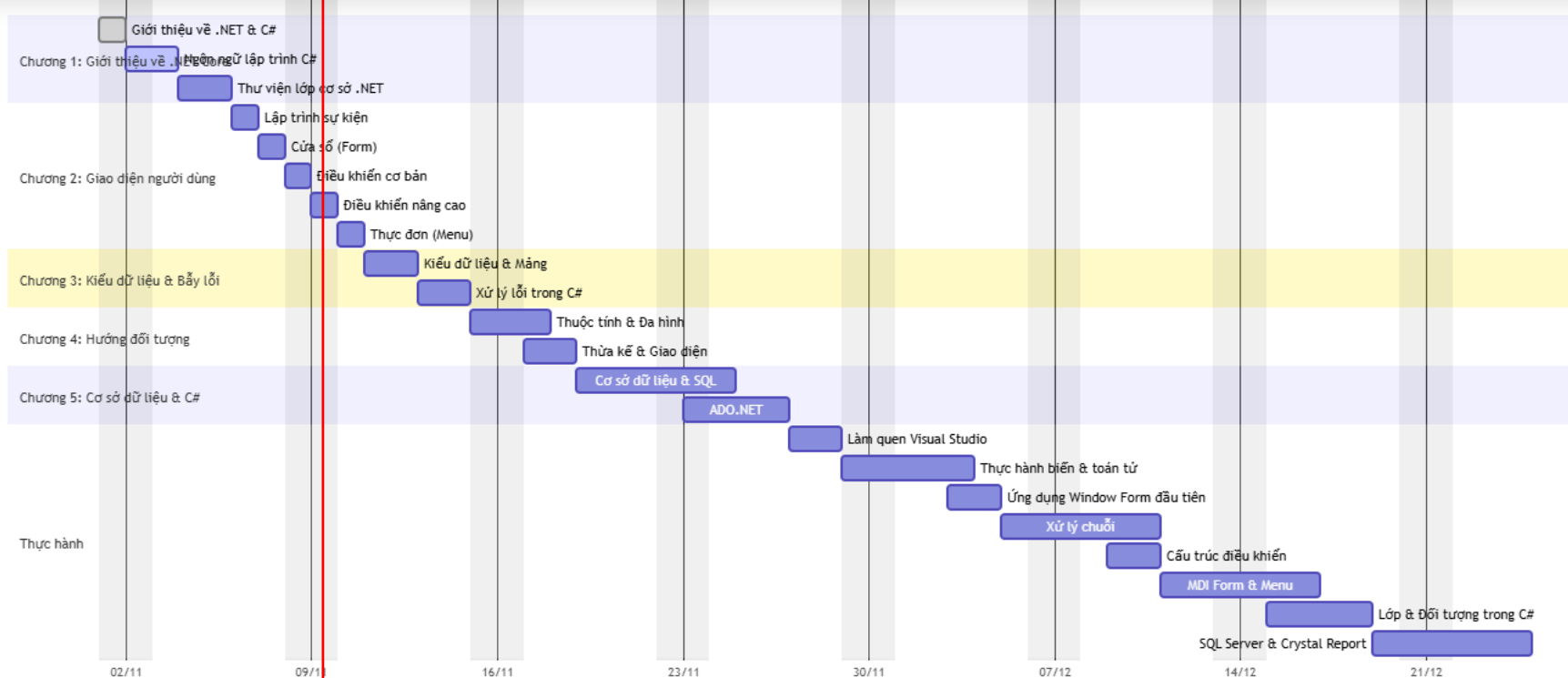
Chuẩn đầu ra: 1, 2

Mục tiêu chương

- Giúp học viên hiểu nền tảng .NET nói chung và .NET Core nói riêng — lịch sử, ưu điểm, và vai trò của nó trong phát triển ứng dụng hiện đại.
- Cung cấp cái nhìn tổng quan về ngôn ngữ C# — đặc điểm chính, tính năng nổi bật.
- Giới thiệu khái quát về thư viện lớp cơ sở (Base Class Library) trong .NET — các thành phần cơ bản mà lập trình viên thường dùng.

Select all

0 selected



1.1.1 .NET là gì?

- **.NET** (trước đây gồm .NET Framework, .NET Core, Mono, Xamarin...) là một nền tảng phần mềm được Microsoft phát triển để hỗ trợ xây dựng và chạy ứng dụng trên nhiều môi trường khác nhau. [Wikipedia](#) +2 [Microsoft for Developers](#) +2
- Nó bao gồm runtime (máy ảo thực thi mã quản lý), thư viện lớp (Base Class Library), công cụ phát triển (IDE, CLI) và hệ sinh thái hỗ trợ.
- .NET cho phép các ngôn ngữ như C#, F#, VB.NET... chia sẻ cùng môi trường thực thi, cùng thư viện chung (chia sẻ API chuẩn).

Select all

0 selected

- Từ .NET 5 trở đi, Microsoft hợp nhất .NET Core và .NET Framework thành một dòng .NET duy nhất, nhưng bản chất “Core” vẫn là xương sống của nền tảng. [Dot Net Tutorials +2 Microsoft for Developers +2](#)

Visual Studio Installer

InstalledAvailable

Checking for updates...

Visual Studio Build Tools 2022 (2)

17.14.2

Update available

17.14.19View details

Modify

Launch

More

Update

Visual Studio Enterprise 2019

16.11.10

Setup paused

Release notes

Resume

Modify

More

Visual Studio Enterprise 2022

17.14.2

Update available

17.14.19View details

Modify

Launch

More

Update

Visual Studio Community 2019 (2)

16.11.10

Modify

Launch

Developer News

Join us at .NET Conf: Dive into the future of development with Visual Studio 2026

Were thrilled to invite you to one of the most exci...

Friday, October 31, 2025

Visual Studio October Update – new models, memories, planning, and more

The October 2025 update for Visual Studio 2022 (...)

Thursday, October 30, 2025

Introducing Planning in Visual Studio (Public Preview)

Quick prompts are great for small fixes, but they f...

Wednesday, October 22, 2025

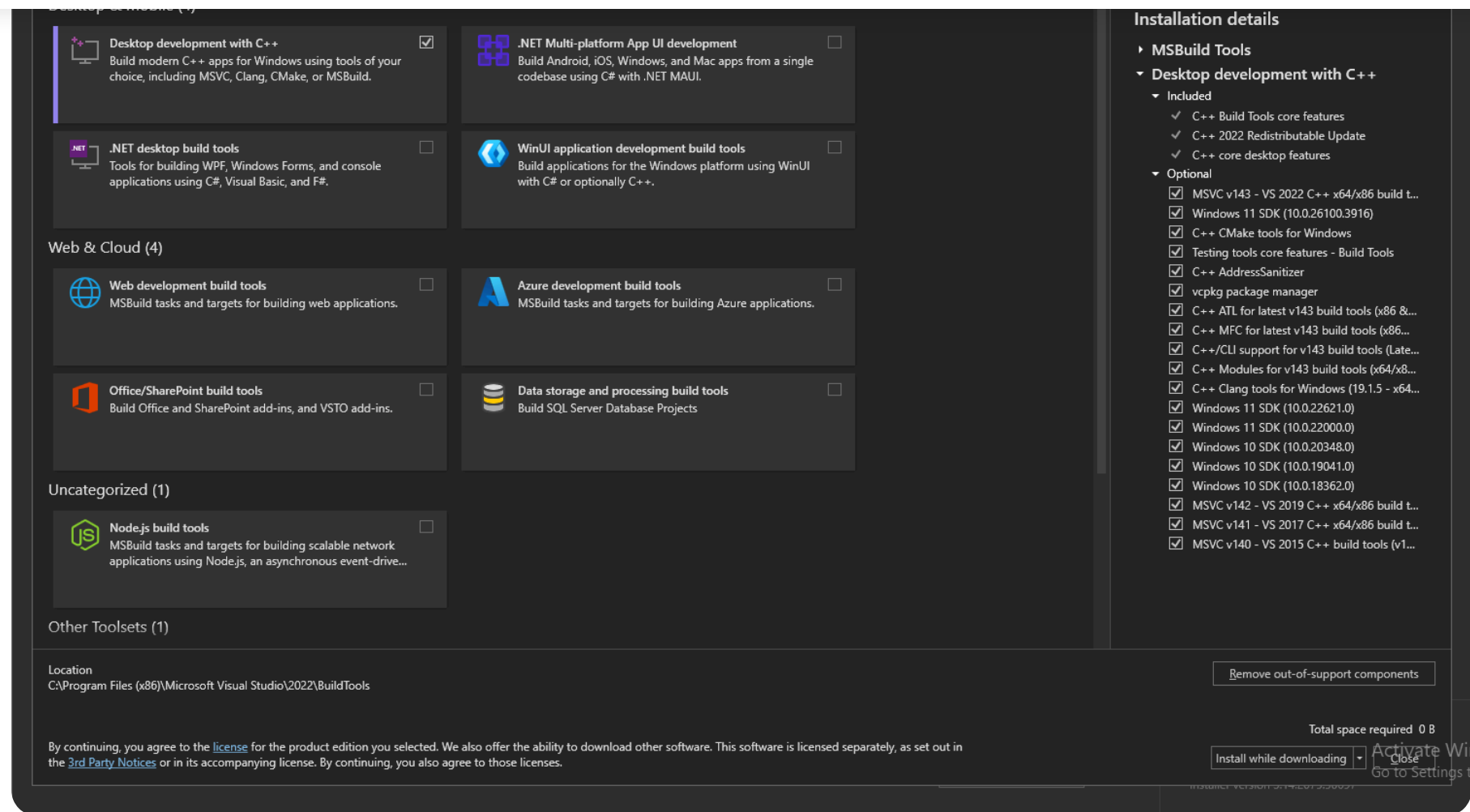
View more Microsoft developer news...

Need help? Check out the Microsoft Developer Community or reach us via Visual Studio Support.

Installer version 3.14.2075.50697

Select all

0 selected



Cài đặt các thành phần của môi trường phát triển VS 2022

Ưu điểm nổi bật của .NET Core / .NET hiện đại:

- **Đa nền tảng (Cross-platform):** Ứng dụng .NET có thể chạy trên Windows, Linux, macOS mà không cần viết lại mã nền tảng riêng. [Microsoft for Developers +2](#) [AddWeb Solution +2](#)

Select all

0 selected

- **Hiệu năng cao và tối ưu:** Runtime (CLR / CoreCLR) được tối ưu, JIT compiler (RyuJIT) và thu gom rác hiện đại giúp ứng dụng nhanh hơn. [Microsoft for Developers +1](#)
- **Hiệu năng cao và tối ưu:** Runtime (CLR / CoreCLR) được tối ưu, JIT compiler (RyuJIT) và thu gom rác hiện đại giúp ứng dụng nhanh hơn. [Microsoft for Developers +2 Dot Net Tutorials +2](#)
- **Công cụ dòng lệnh (CLI) thống nhất:** cho phép xây dựng, biên dịch, đóng gói ứng dụng từ terminal trên mọi nền tảng. [Microsoft for Developers](#)
- **Triển khai linh hoạt:** Có hai cách triển khai chính — phụ thuộc runtime (Framework-dependent) hoặc đóng gói toàn bộ runtime (Self-contained) — giúp quản lý phiên bản dễ hơn. [Microsoft for Developers +1](#)

Một số lưu ý lịch sử / định hướng:

- .NET Core 1.0 ra mắt vào tháng 6/2016. [C# Corner +1](#)
- .NET Core 3.1 là phiên bản LTS (Long-Term Support). [SOFTACOM +1](#)
- Từ .NET 5 trở đi, tên “Core” không còn được dùng, nhưng nền tảng vẫn kế thừa các đặc tính của .NET Core. [Dot Net Tutorials +1](#)

1.2 Ngôn ngữ lập trình C#

1.2.1 Giới thiệu C#

- **C#** là ngôn ngữ lập trình hướng đối tượng do Microsoft phát triển, chạy trên nền tảng .NET.
- C# thiết kế để đơn giản, hiện đại, an toàn và linh hoạt — kết hợp các đặc điểm từ C, C++ và Java, thêm các tính năng mới như LINQ, biểu thức Lambda, Async/Await, kiểu Nullable, tuple, pattern matching... [ankibooks.io +2 C# Corner +2](#)

Select all

0 selected

C# chạy trong môi trường quản lý (managed environment), nó cung cấp các tính năng như quản lý bộ nhớ tự động (garbage collection), kiểm soát bảo mật và sandboxing.

1.2.2 Các tính năng nổi bật của C#

Một số tính năng mà bạn nên giới thiệu khi dạy:

- **Hướng đối tượng đầy đủ:** lớp, kế thừa, interface, đa hình, đóng gói.



```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        int[] numbers = { 2, 7, 1, 9, 4, 6, 8 };

        // Truy vấn LINQ: chọn các số chẵn và sắp xếp tăng dần
        var evenNumbers = from n in numbers
                           where n % 2 == 0
                           orderby n
                           select n;

        Console.WriteLine("Các số chẵn:");
        foreach (var num in evenNumbers)
            Console.WriteLine(num);
    }
}
```

- **Async / Await:** lập trình bất đồng bộ để đọc, tránh blocking UI hoặc thread.
- **Delegates, Events và Lambda Expressions:** giúp xử lý callback, sự kiện, biểu thức ngắn gọn.
- **Nullable Types & Null safety:** hỗ trợ kiểu dữ liệu có thể null và các toán tử an toàn.
- **Pattern Matching:** cú pháp để kiểm tra kiểu và cấu trúc dữ liệu.
- **Tuples và deconstruction:** để trả nhiều giá trị dễ dàng, destructuring dữ liệu.

Select all

0 selected

1.3.1 Khái niệm và vai trò

- **Base Class Library (BCL)** là tập hợp các lớp, interface, kiểu dữ liệu cơ bản mà mọi ứng dụng .NET đều có thể dùng. Nó cung cấp hầu hết các chức năng nền tảng như thao tác chuỗi, xử lý file, cấu trúc dữ liệu, thread, IO, net, v.v. [Microsoft Learn +2](#) [Microsoft for Developers +2](#)
- BCL là phần của .NET Standard / .NET Core và các ngôn ngữ .NET đều truy cập được những API này. [Microsoft for Developers +1](#)

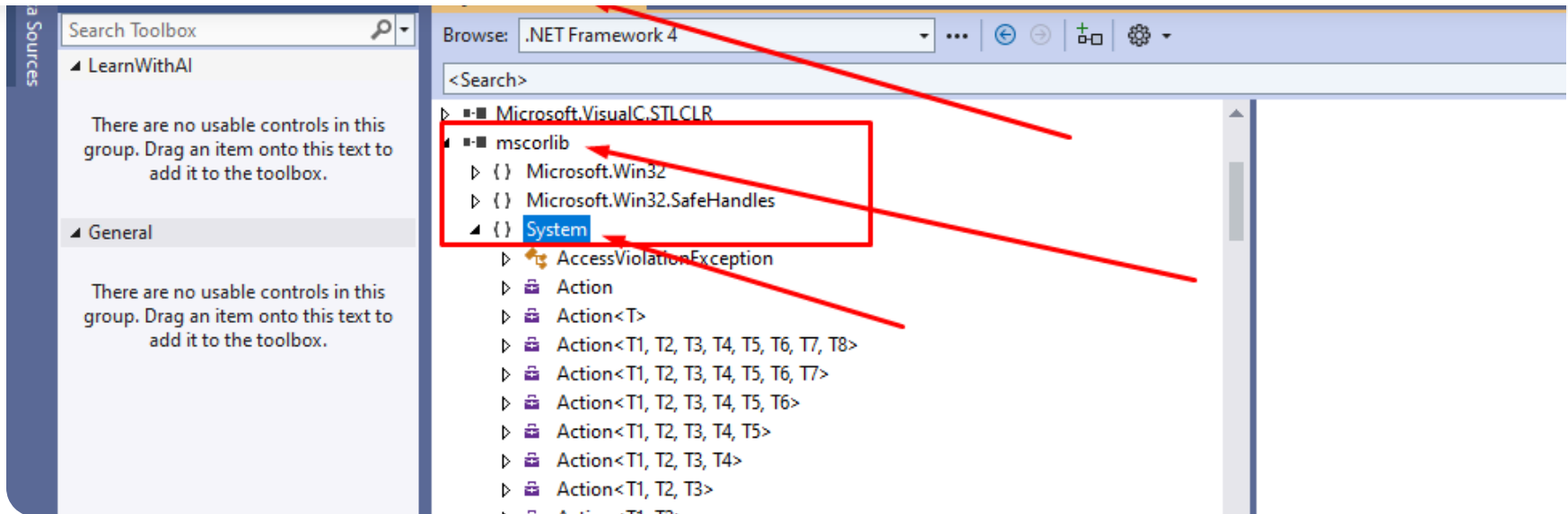
1.3.2 Các thành phần thường dùng

Một số namespace và lớp phổ biến trong BCL mà bạn nên giới thiệu:

- **System:** chứa các kiểu dữ liệu cơ bản như `Object`, `String`, `Int32`, `Array`, `Console`, `Exception`... [Microsoft Learn +1](#)

Select all

0 selected



- **System.Collections / System.Collections.Generic:** các cấu trúc dữ liệu như `List<T>`, `Dictionary<TKey, TValue>`, `Queue`, `Stack`.
- **System.IO:** để làm việc với file, thư mục, stream (`FileStream`, `StreamReader`, `StreamWriter`).
- **System.Threading / System.Threading.Tasks:** để làm việc đa luồng, Task / async programming.
- **System.Net / System.Net.Http:** để làm việc với mạng, giao tiếp HTTP, `WebClient`, `HttpClient`.
- **System.Text / System.Text.RegularExpressions:** xử lý văn bản, chuỗi, mã hoá, biểu thức chính quy.
- **System.Linq:** hỗ trợ truy vấn dữ liệu trong bộ sưu tập nhờ LINQ.
- **System.Diagnostics:** logging, đo thời gian, theo dõi.
- **System.Security:** các API bảo mật như mã hoá, chứng thực, quyền truy cập.

1.3.3 BCL & .NET Standard

Select all

0 selected

- BCL là một phần của .NET Standard — nghĩa là các lớp BCL cơ bản luôn có trong mọi phiên bản .NET.

Gợi ý tổ chức tiết học & hoạt động

- Mở đầu: giải thích khái niệm .NET, sự chuyển đổi từ .NET Framework sang .NET Core / .NET hiện đại.
- Giải thích C# — từ khái niệm đến các tính năng nổi bật, cùng các đoạn mã mẫu nhỏ.
- Giới thiệu BCL qua ví dụ: dùng `System.IO`, `List<T>`, `HttpClient` để làm việc file và mạng.
- Thảo luận ưu điểm của .NET Core (đa nền tảng, hiệu năng, modular...) so với .NET Framework truyền thống.
- Bài tập nhóm: so sánh dự án mẫu giữa .NET Framework và .NET Core (cấu trúc thư mục, file csproj, cách deploy).
- Thảo luận xu hướng hiện nay: .NET 5 / 6 / 7, kế hoạch tương lai của .NET.



Chương 2 Giao diện người dùng (User Interface)



Trong chương này, học viên sẽ được làm quen với cách tạo giao diện người dùng trong ứng dụng C#, hiểu mô hình lập trình sự kiện, cách làm việc với biểu mẫu (form), và sử dụng các điều khiển cơ bản, điều khiển nâng cao cũng như thực đơn (menu).

Select all

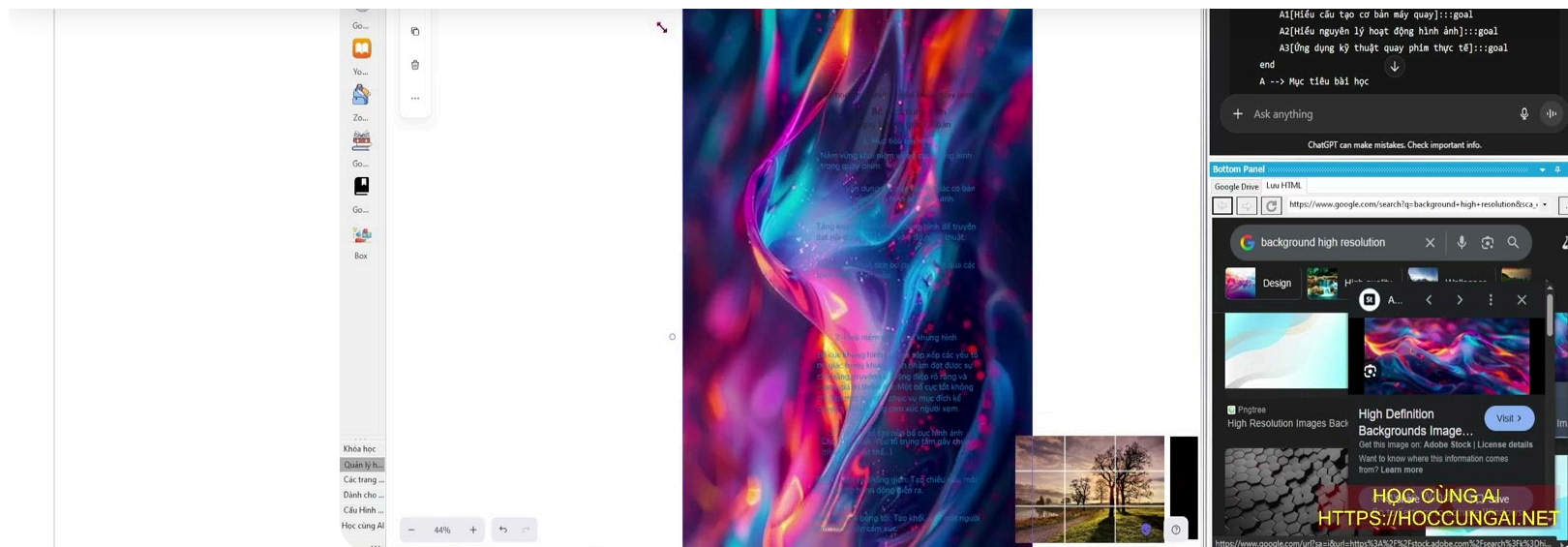
0 selected

**BÀI HƯỚNG DẪN
CÁC BƯỚC TRIỂN KHAI
MỘT ỨNG DỤNG. MỞ MÃ.
VÍ DỤ APP THI HÀNH CODE PYTHON**

Triển khai ứng dụng

Select all

0 selected



Ứng dụng bố cục trong thiết kế UI

2.1 Lập trình sự kiện (Event Programming)

Khái niệm

- Trong lập trình giao diện người dùng (GUI), mô hình chủ đạo là **lập trình sự kiện** (event-driven programming).
- Ý tưởng cơ bản: giao diện sẽ “ngồi chờ” (listening) các sự kiện từ người dùng như nhấn nút, di chuyển chuột, nhập bàn phím, thay đổi giá trị điều khiển... Khi sự kiện xảy ra, hệ thống sẽ gọi (invoke) hàm xử lý sự kiện (event handler) để thực thi logic tương ứng.
- Điều này trái với mô hình tuần tự – chương trình không “chạy từ đầu đến cuối” mà phản ứng theo sự kiện.

Cấu trúc lập trình sự kiện trong C#

1. **Sự kiện (Event):** Là cái gì đó “xảy ra”, ví dụ Button.Click, MouseMove, KeyDown.

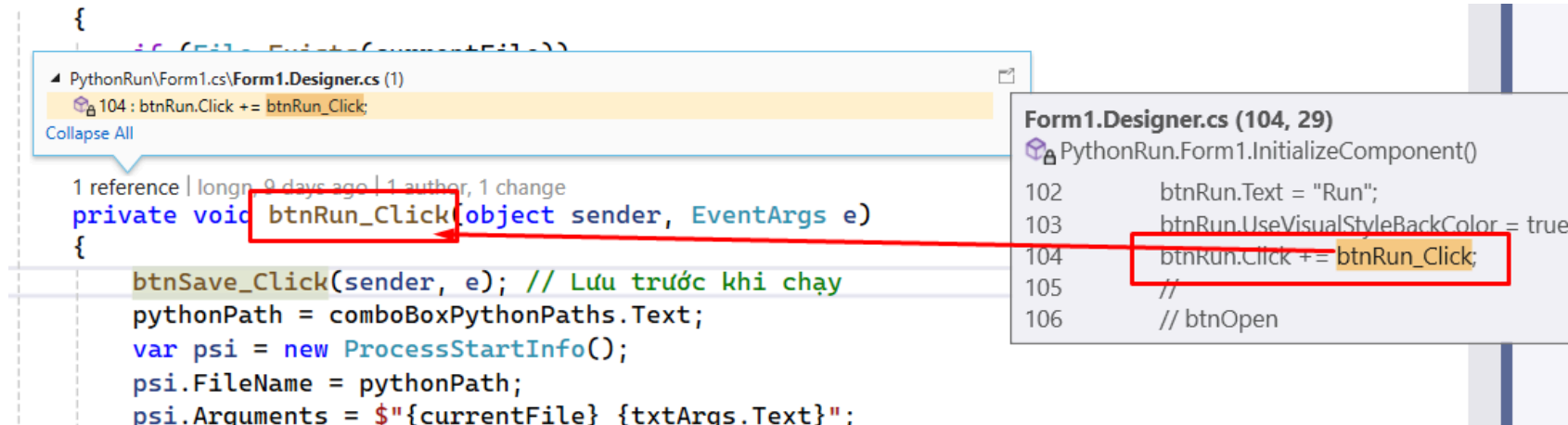
Select all

0 selected

3. **Event Handler (Hàm xử lý sự kiện):** Là phương thức (hàm) được đăng ký với sự kiện để thực thi khi sự kiện được kích hoạt.

4. **Đăng ký sự kiện (Subscription):** Sự kiện và handler cần được liên kết – có thể bằng cú pháp như `button.Click += MyButtonClick;`

5.



```
{
    // File: Form1.Designer.cs
    PythonRun\Form1.cs\Form1.Designer.cs (1)
    104 : btnRun.Click += btnRun_Click;
    Collapse All

    1 reference | longn 9 days ago | 1 author, 1 change
    private void btnRun_Click(object sender, EventArgs e)
    {
        btnSave_Click(sender, e); // Lưu trước khi chạy
        pythonPath = comboBoxPythonPaths.Text;
        var psi = new ProcessStartInfo();
        psi.FileName = pythonPath;
        psi.Arguments = $"{currentFile} {txtArgs.Text}";
    }
}
```

Form1.Designer.cs (104, 29)

```
PythonRun.Form1.InitializeComponent()
102     btnRun.Text = "Run";
103     btnRun.UseVisualStyleBackColor = true;
104     btnRun.Click += btnRun_Click;
105     //
106     // btnOpen
```

6. **Hủy đăng ký sự kiện (Unsubscription):** Khi không cần nữa, có thể bỏ liên kết để tránh rò rỉ bộ nhớ, như `button.Click -= MyButtonClick;`

Ví dụ minh họa

```
public partial class MainForm : Form { public MainForm() { InitializeComponent(); // Đăng ký sự kiện
this.buttonHello.Click += ButtonHello_Click; } private void ButtonHello_Click(object sender, EventArgs e) {
MessageBox.Show("Xin chào! Bạn đã nhấn nút."); } }
```

- `buttonHello.Click` là sự kiện.
- `ButtonHello_Click` là hàm xử lý sự kiện.

Ghi chú & best practices

- Luôn kiểm tra **sender** hoặc cast nó nếu cần biết điều khiển nào phát sự kiện.
- Cố gắng giữ event handler ngắn gọn — nếu logic phức tạp, gọi hàm khác từ trong handler.
- Hủy đăng ký sự kiện khi form / điều khiển bị dispose để tránh memory leak.

2.2 Cửa sổ (Biểu mẫu, Form)

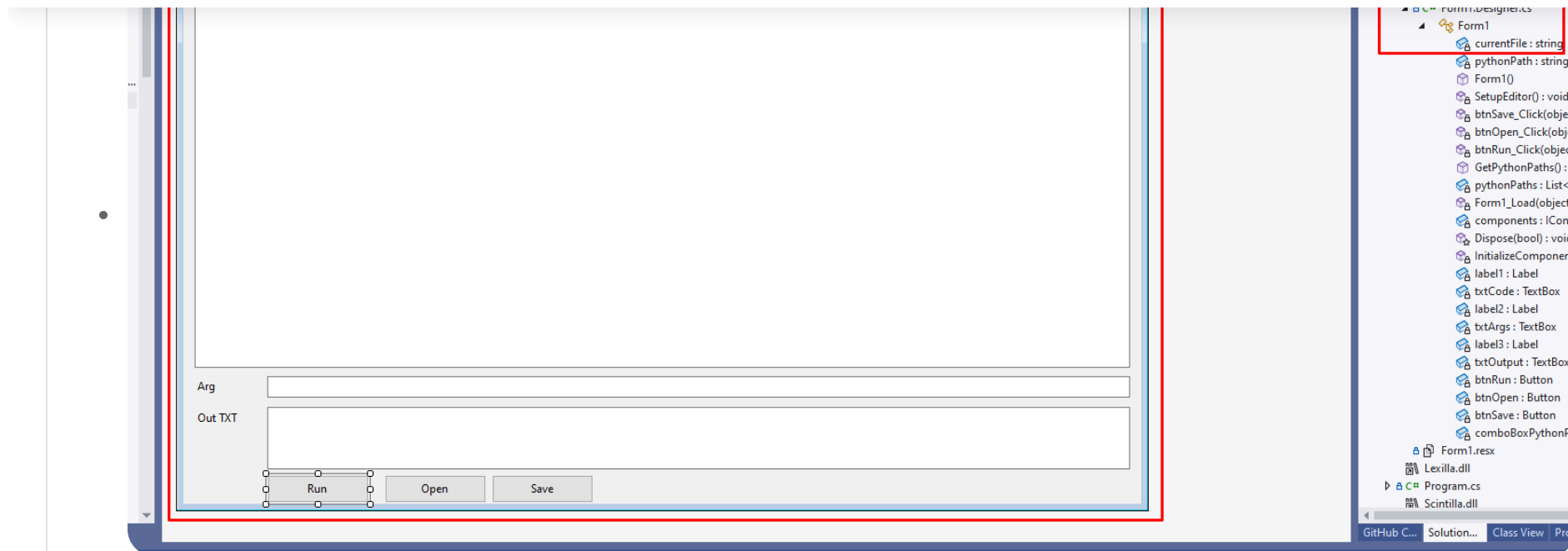
Trong ứng dụng Windows Forms hoặc ứng dụng desktop .NET, **Form** (biểu mẫu) là đối tượng cơ bản đại diện cho cửa sổ ứng dụng.

Khái niệm Form / Biểu mẫu

- Một *Form* là một cửa sổ GUI chứa các điều khiển (button, textbox, checkbox...).
- Form chứa vùng client (nơi đặt điều khiển), viền, thanh tiêu đề, các nút điều khiển (thu nhỏ, đóng, phóng đại).
- Trong WPF, bạn có **Window** tương đương. Nhưng trong Windows Forms (WinForms) hay .NET Core WinForms, **Form** rất phổ biến.

Select all

0 selected



Các thuộc tính quan trọng của Form

- **Text**: tiêu đề của form hiển thị trên thanh tiêu đề.
- **Size / Width / Height**: kích thước cửa sổ.
- **StartPosition**: vị trí xuất hiện cửa sổ khi khởi động (ví dụ: CenterScreen).
- **FormBorderStyle**: kiểu khung cửa sổ (FixedDialog, Sizable, None ...).
- **BackColor / ForeColor**: màu nền hoặc màu chữ mặc định.
- **Icon**: biểu tượng cửa sổ.
- Sự kiện quan trọng của Form: **Load**, **Shown**, **FormClosing**, **FormClosed**, **Resize**.

```
public partial class MyForm : Form { public MyForm() { InitializeComponent(); this.Text = "Ứng dụng mẫu";  
this.StartPosition = FormStartPosition.CenterScreen; this.FormClosing += MyForm_FormClosing; } private void  
MyForm_FormClosing(object sender, FormClosingEventArgs e) { var result = MessageBox.Show("Bạn muốn thoát?",  
"Xác nhận", MessageBoxButtons.YesNo); if (result == DialogResult.No) { e.Cancel = true; } } }
```

Ở ví dụ trên: Khi người dùng cố gắng đóng form, sự kiện **FormClosing** được kích hoạt để hỏi xác nhận.

2.3 Các điều khiển cơ bản trong C#

Form chỉ là “vỏ” — nội dung tương tác là các **điều khiển (controls)**. Dưới đây là các điều khiển cơ bản mà bạn nên giới thiệu:

Một số điều khiển phổ biến

- **Button** – nút bấm, phát sinh sự kiện **Click**.
- **Label** – nhãn hiển thị văn bản, không tương tác.
- **TextBox** – hộp nhập văn bản đơn dòng hoặc đa dòng (**Multiline = true**).
- **CheckBox** – hộp kiểm (true/false).
- **RadioButton** – lựa chọn giữa nhiều lựa chọn (thuộc nhóm).
- **ComboBox** – danh sách thả xuống, cho phép chọn một mục hoặc nhập.
- **ListBox** – danh sách nhiều mục, có thể chọn nhiều hoặc đơn.
- **DataGridView** – bảng dữ liệu, hiển thị và có khả năng chỉnh sửa hàng/cột.
- **PictureBox** – hiển thị hình ảnh.

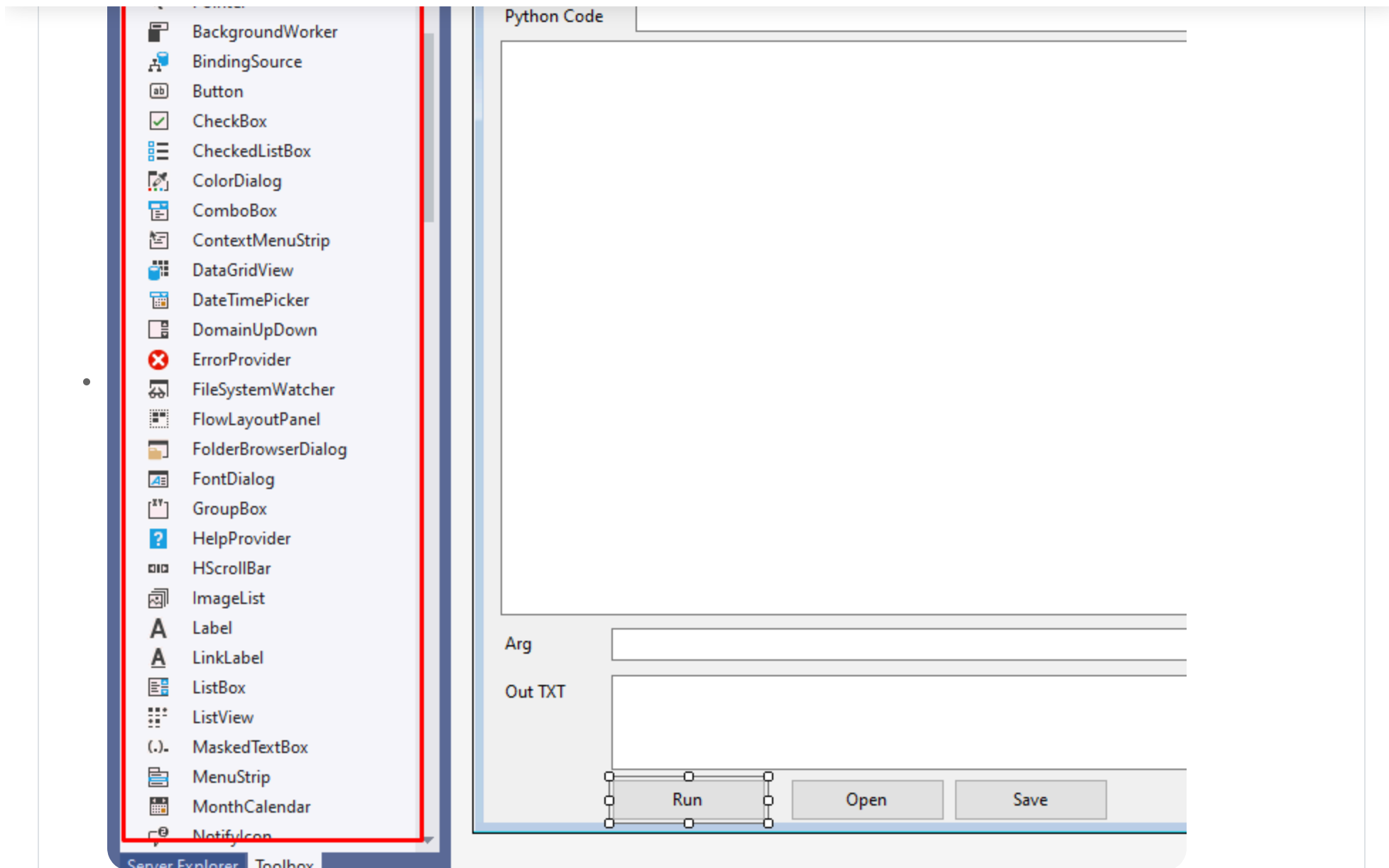
Select all

0 selected

- **Panel / GroupBox** – container để nhóm các điều khiển nhỏ lại, hỗ trợ bố cục.

Select all

0 selected



Ví dụ sử dụng TextBox và Button

Select all

0 selected

```
buttonSubmit đã đặt trong Designer buttonSubmit.Click += ButtonSubmit_Click; } private void  
ButtonSubmit_Click(object sender, EventArgs e) { string name = textBoxName.Text; if  
(string.IsNullOrEmpty(name)) { MessageBox.Show("Xin nhập tên!"); } else { MessageBox.Show($"Chào bạn,  
{name}!"); } } }
```

Lưu ý khi dùng điều khiển

- **Kiểm tra dữ liệu nhập vào (validation).**
- Thông báo lỗi rõ ràng cho người dùng.
- Sắp xếp và bố trí hợp lý (layout) để giao diện dễ dùng.
- Gán giá trị mặc định nếu cần.

2.4 Các điều khiển nâng cao

Ngoài các điều khiển cơ bản, còn có nhiều điều khiển nâng cao giúp tạo giao diện phong phú, linh hoạt hơn:

- **TreeView / ListView** – hiển thị dữ liệu phân cấp hoặc dữ liệu dạng danh sách chi tiết.
- **TabControl** – tạo các tab để phân chia nội dung trong cùng một form.
- **SplitContainer** – phân chia form thành hai phần có thể kéo thay đổi kích thước.
- **ToolStrip / Toolbar** – thanh công cụ, nút tắt nhanh.
- **StatusStrip / StatusBar** – hiển thị thông tin trạng thái ở đáy cửa sổ.
- **ToolTip** – hiển thị gợi ý nhỏ khi người dùng hover chuột vào điều khiển.
- **DateTimePicker / TimePicker** – chọn ngày / giờ.

- **WebBrowser Control** – hiển thị nội dung HTML bên trong ứng dụng WinForms (nếu cần nhúng Web).

Ví dụ: Bạn có thể dùng **TabControl** để chia giao diện thành các tab “Thông tin”, “Cài đặt”, “Báo cáo”.

2.5 Thực đơn (Menu)

Thực đơn là thành phần quan trọng trong nhiều ứng dụng desktop – giúp người dùng truy cập các tính năng qua menu bar hoặc context menu.

Các loại menu

- **MenuStrip / MainMenu** – thanh menu nằm ngang phía trên form, chứa các mục như File, Edit, View...
- **ContextMenuStrip / ContextMenu** – menu ngữ cảnh bật lên khi click chuột phải vào điều khiển hoặc form.
- **ToolStripMenuItem** – mục menu đơn lẻ, có thể chứa các menu con (submenu).
- **Separator** – dấu phân cách giữa các mục menu.
- **Shortcut keys / hotkeys** – phím tắt gán cho mục menu, ví dụ Ctrl+S để “Save”.

Ví dụ tạo menu cơ bản

```
public partial class MenuForm : Form { public MenuForm() { InitializeComponent(); // Tạo MenuStrip trong code
(có thể làm trong Designer) MenuStrip menu = new MenuStrip(); ToolStripMenuItem fileMenu = new
ToolStripMenuItem("File"); ToolStripMenuItem exitItem = new ToolStripMenuItem("Thoát"); exitItem.ShortcutKeys
= Keys.Control | Keys.X; exitItem.Click += (s, e) => this.Close(); fileMenu.DropDownItems.Add(exitItem);
menu.Items.Add(fileMenu); this.MainMenuStrip = menu; this.Controls.Add(menu); } }
```


Lưu ý khi thiết kế menu

- Sắp xếp menu theo logic, nhóm chức năng liên quan lại với nhau.
- Hạn chế số mục quá lớn, tránh làm rối giao diện.
- Gán phím tắt phổ biến để tăng trải nghiệm người dùng.
- Cập nhật trạng thái menu (Enabled/Disabled) tùy theo trạng thái ứng dụng.

Gợi ý triển khai bài giảng và thực hành

- Bắt đầu với mô hình sự kiện — minh họa bằng ví dụ nhỏ: nhấn nút, thay đổi giá trị điều khiển.
- Cho học viên thiết kế form đơn giản — chứa Label, TextBox, Button, thử xử lý sự kiện nhập tên + chào hỏi.
- Giới thiệu các điều khiển cơ bản khác — đặt vào form, cấu hình thuộc tính, xử lý sự kiện của chúng.
- Nâng cao bằng việc thêm TabControl, TreeView hoặc ListView vào form ví dụ.
- Cuối cùng, hướng dẫn làm menu bằng MenuStrip + ToolStripMenuItem, gán sự kiện và phím tắt.
- Bài tập thực hành: xây dựng ứng dụng nhỏ “Danh bạ người dùng” có giao diện form, danh sách (ListView), menu để thêm/sửa/xóa, form nhập dữ liệu.



Select all

0 selected

- Nam vùng các kiểu dữ liệu cơ bản trong C#
- Biết cách khai báo và sử dụng biến, hằng số

❖ Nội dung chính

1. Phân loại kiểu dữ liệu trong C#

Phân loại	Mô tả	Ví dụ
Kiểu giá trị	Lưu trữ trực tiếp giá trị	<code>int, double, bool, char, struct</code>
Kiểu tham chiếu	Lưu địa chỉ tới vùng nhớ	<code>string, object, class, array</code>
Nullable types	Cho phép kiểu giá trị có giá trị null	<code>int?, bool?</code>

2. Các kiểu dữ liệu phổ biến

- Số nguyên: `int, short, long, byte`
- Số thực: `float, double, decimal`
- Ký tự: `char`
- Chuỗi: `string`
- Luận lý: `bool`
- Đối tượng: `object`

```
int age = 20; double salary = 1500.50; bool isActive = true; string name = "Học viên";
```

3. Hằng số

```
const double Pi = 3.14; readonly int MaxUsers = 100; // trong class
```

4. Ép kiểu

- **Tường minh (explicit):**

```
double d = 9.8; int i = (int)d; // Kết quả: 9
```

- **Ngầm định (implicit):**

```
int x = 10; double y = x; // Không cần ép kiểu
```

3.2 Kiểu liệt kê, mảng và tập hợp (2 tiết)

❖ Mục tiêu

- Biết cách định nghĩa và sử dụng **enum**
- Làm việc với **mảng** và **các kiểu tập hợp (List, Dictionary...)**

❖ Nội dung chính

1. Kiểu liệt kê (enum)

- Dùng để định nghĩa một tập các hằng số có tên

2. Mảng (Array)

- Khai báo:

```
int[] numbers = new int[5]; // Mảng 5 phần tử string[] names = { "An", "Bình", "Cường" };
```

- Duyệt mảng:

```
foreach (string name in names) { Console.WriteLine(name); }
```

- Chỉ số bắt đầu từ 0 – Cần kiểm tra giới hạn tránh lỗi `IndexOutOfRangeException`.

3. Tập hợp (Collections)

a. List<T>

- Mảng động, linh hoạt hơn mảng thông thường

```
List<string> students = new List<string>(); students.Add("Lan"); students.Add("Minh");
```

b. Dictionary<TKey, TValue>

- Tập hợp các cặp khóa – giá trị

Select all

0 selected

c. **HashSet<T>** – tập hợp không lặp lại

d. **Queue<T>**, **Stack<T>**

- **Queue**: FIFO – hàng đợi
- **Stack**: LIFO – ngăn xếp

3.3 Xử lý lỗi trong C# (Exception Handling) (2 tiết)

❖ Mục tiêu

- Biết cách bắt lỗi và xử lý ngoại lệ trong quá trình thực thi chương trình

❖ Nội dung chính

1. Ngoại lệ (Exception) là gì?

- Là sự kiện xảy ra trong quá trình chạy khiến chương trình không thể tiếp tục bình thường.
- Ví dụ: chia cho 0, truy cập mảng ngoài phạm vi, null reference, định dạng sai...

2. Cấu trúc try – catch – finally

```
try { int x = int.Parse("abc"); // Gây lỗi FormatException } catch (FormatException ex) {  
    Console.WriteLine("Lỗi định dạng số."); } catch (Exception ex) { Console.WriteLine("Lỗi chung: " +  
    ex.Message); } finally { Console.WriteLine("Khối finally luôn được thực hiện."); }
```

3. Một số loại ngoại lệ thường gặp

Ngoại lệ	Mô tả
<code>NullReferenceException</code>	Tham chiếu đến đối tượng null

Select all

0 selected

DivideByZeroException	Chia số cho 0
IOException	Lỗi nhập/xuất

4. Tự tạo ngoại lệ (custom exception)

```
public class AgeException : Exception { public AgeException(string message) : base(message) { } }
```

5. Dùng throw để ném lỗi

```
if (age < 0) throw new AgeException("Tuổi không thể âm!");
```

6. Best practices

- Luôn **bắt lỗi cụ thể trước**, lỗi chung sau.
- **Không bắt lỗi nếu không xử lý được.**
- **Không giấu lỗi** bằng catch trống.
- Ghi log hoặc thông báo lỗi rõ ràng cho người dùng.

Bài tập thực hành gợi ý

1. Viết chương trình nhập tên và tuổi, kiểm tra:

- Tuổi phải là số nguyên dương.
- Nếu nhập sai thì xử lý lỗi và in thông báo rõ ràng.

Select all

0 selected

3. Tạo `List<string>` lưu tên sinh viên. Thêm, xóa, tìm kiếm và in danh sách.

4. Tạo chương trình chia hai số. Nếu chia cho 0, phải báo lỗi “Không thể chia cho 0”.



☒ Chương 4 Lập Trình Hướng Đối Tượng với C#



(Ôn tập lại môn học lập trình hướng đối tượng)

4.1 Thuộc tính (Properties) (1 tiết)

❖ Mục tiêu

- Hiểu cách hoạt động của **thuộc tính (property)** trong C#
- Biết cách sử dụng thuộc tính để bảo vệ dữ liệu (encapsulation)

❖ Nội dung chính

1. Khái niệm

- Property là cách tiếp cận **an toàn** để truy cập hoặc gán giá trị cho một **trường dữ liệu riêng (private)** của class.
- Property kết hợp giữa **get** và **set**.

2. Cú pháp

Select all

0 selected

3. Auto-implemented Property

```
public class Student { public string Name { get; set; } }
```

4. Property chỉ đọc hoặc chỉ ghi

```
public int Age { get; private set; } // chỉ đọc từ bên ngoài
```


Select all

0 selected

3 references | longn, 9 days ago | 1 author, 1 change

`public partial class Form1 : Form`

`{`

`private readonly string currentFile = @"script.py"; // ID`

`private string pythonPath; // CS0236 fix: Moved initializ`

1 reference | longn, 9 days ago | 1 author, 1 change

`public Form1()`

`{`

`InitializeComponent();`

`SetupEditor();`

`// Initialize pythonPath after comboBoxPythonPaths is`

`pythonPath = comboBoxPythonPaths.Text;`

`}`

1 reference | longn, 9 days ago | 1 author, 1 change

`private void SetupEditor()`

5. Property với logic trong get/set

```
private int age; public int Age { get { return age; } set { if (value >= 0 && value <= 120) age = value; } }
```

4.2 Đa hình (Polymorphism) (1 tiết)

❖ Mục tiêu

❖ Nội dung chính

1. Khái niệm

- Đa hình cho phép **một đối tượng** có thể thực hiện **nhiều hành vi khác nhau** dựa trên ngữ cảnh.

2. Đa hình ở **compile-time** (nạp chồng phương thức - method overloading)

```
public class Calculator { public int Add(int a, int b) => a + b; public double Add(double a, double b) => a + b; }
```

3. Đa hình ở **runtime** (ghi đè phương thức - method overriding)

```
class Animal { public virtual void Speak() { Console.WriteLine("Animal speaks"); } } class Dog : Animal { public override void Speak() { Console.WriteLine("Dog barks"); } }
```

4. Từ khóa **virtual**, **override**, **abstract**

- **virtual**: phương thức có thể ghi đè
- **override**: ghi đè phương thức ở lớp con
- **abstract**: buộc lớp con phải triển khai

4.3 Thừa kế (Inheritance) (1 tiết)

❖ Mục tiêu

❖ Nội dung chính

1. Khái niệm

- Cho phép lớp con kế thừa **thuộc tính và phương thức** từ lớp cha.

2. Cú pháp

```
class Person { public string Name { get; set; } } class Student : Person { public string StudentID { get; set; } }
```

3. Từ khóa **base**

- Dùng để gọi constructor hoặc phương thức của lớp cha.

```
class Student : Person { public Student(string name) : base(name) { } }
```

4. Các lưu ý

- C# không hỗ trợ đa kế thừa trực tiếp giữa các lớp.
- Có thể sử dụng **giao diện** để đạt đa kế thừa chức năng.

4.4 Giao diện (Interface) (2 tiết)

❖ Mục tiêu

- Hiểu khái niệm và vai trò của **interface**

1. Khái niệm

- Interface là một tập hợp các khai báo (thuộc tính, phương thức, sự kiện...) **mà không có phần thân (implementation)**.
- Lớp hoặc struct **phải triển khai đầy đủ** các thành phần của interface nếu khai báo thực hiện interface.

2. Khai báo interface

```
public interface IAnimal { void Speak(); }
```

3. Triển khai interface

```
public class Dog : IAnimal { public void Speak() { Console.WriteLine("Gâu gâu!"); } }
```

4. Giao diện với thuộc tính

```
public interface IStudent { string Name { get; set; } }
```

5. Một lớp triển khai nhiều giao diện

```
public interface IMoveable { void Move(); } public interface ISpeakable { void Speak(); } public class Robot : IMoveable, ISpeakable { public void Move() => Console.WriteLine("Robot moving"); public void Speak() =>
```

Select all

0 selected

Tiêu chí	Interface	Abstract Class
Kế thừa	Đa kế thừa	Chỉ một
Thành phần triển khai sẵn	Không	Có thể có
Tính trừu tượng	100%	Có thể không 100%

✅ Bài tập đề xuất cho chương 4 (Mấy cái bài tập này AI nó giải 10 giây - nhiệm vụ của sinh viên là đọc hiểu, code lại để ghi nhớ cách thức mô tả chứ giờ không ai code từng dòng):

1. Tạo một lớp `Animal`, có phương thức `Speak()`, sau đó tạo lớp `Dog` và `Cat` kế thừa, ghi đè phương thức `Speak()`.

Select all

0 selected

csharp

 Copy code

```
using System;

namespace AnimalExample
{
    // Lớp cơ sở
    class Animal
    {
        public virtual void Speak()
        {
            Console.WriteLine("Animal makes a sound");
        }
    }

    // Lớp Dog kế thừa Animal
    class Dog : Animal
    {
        public override void Speak()
        {
            Console.WriteLine("Dog says: Woof!");
        }
    }

    // Lớp Cat kế thừa Animal
    class Cat : Animal
    {
        public override void Speak()
        {
            Console.WriteLine("Cat says: Meow!");
        }
    }
}
```

Select all

0 selected

```
using System;

namespace AnimalExample
{
    // Lớp cơ sở
    class Animal
    {
        public virtual void Speak()
        {
            Console.WriteLine("Animal makes a sound");
        }
    }

    // Lớp Dog kế thừa Animal
    class Dog : Animal
    {
        public override void Speak()
        {
            Console.WriteLine("Dog says: Woof!");
        }
    }

    // Lớp Cat kế thừa Animal
    class Cat : Animal
    {
        public override void Speak()
        {
```

```
class Program
{
    static void Main(string[] args)
    {
        Animal myAnimal = new Animal();
        myAnimal.Speak(); // Animal makes a sound

        Dog myDog = new Dog();
        myDog.Speak(); // Dog says: Woof!

        Cat myCat = new Cat();
        myCat.Speak(); // Cat says: Meow!

        // Sử dụng đa hình
        Animal animalDog = new Dog();
        animalDog.Speak(); // Dog says: Woof!

        Animal animalCat = new Cat();
        animalCat.Speak(); // Cat says: Meow!
    }
}
```

1. Viết interface **ICalculator** với các phương thức **Add**, **Subtract**, **Multiply**, **Divide**. Tạo lớp **SimpleCalculator** triển khai interface này.
2. Viết class **Rectangle** có thuộc tính **Width**, **Height**, sử dụng **property có kiểm tra dữ liệu**.

Select all

0 selected



✓ Chương 5 Cơ sở dữ liệu & C# ✎



5.1 Một số khái niệm về cơ sở dữ liệu (2 tiết)

Mục tiêu

- Hiểu được các khái niệm cơ bản của hệ quản trị cơ sở dữ liệu (DBMS) và cơ sở dữ liệu (CSDL)
- Nắm được vai trò và cấu trúc của cơ sở dữ liệu trong phát triển phần mềm

Nội dung chính

1. Khái niệm cơ sở dữ liệu (Database)

- CSDL là tập hợp có cấu trúc của dữ liệu được lưu trữ và quản lý một cách có hệ thống.
- Các loại CSDL: quan hệ, phi quan hệ (NoSQL), đa chiều, phân tán, v.v.

2. Hệ quản trị cơ sở dữ liệu (DBMS)

- DBMS là phần mềm để tạo, quản lý, truy cập và bảo mật cơ sở dữ liệu.
- Các hệ quản trị phổ biến: SQL Server, MySQL, Oracle, PostgreSQL, MongoDB...

Select all

0 selected

The screenshot shows the SQLite Browser application interface. On the left, a tree view displays the database structure, including a table named 'Code'. The main window is the 'CREATE TABLE' dialog for the 'Code' table. The 'Fields' tab is active, showing a list of fields with their types and constraints. The 'Code' table has the following fields:

Name	Type	NN	PK	AI	U	Default	Check
ID	INTEGER	☒	☒	☒	☐		
FileName	TEXT	☐	☐	☐	☐		
Description	TEXT	☐	☐	☐	☐		
PythonCode	TEXT	☐	☐	☐	☐		
Args	TEXT	☐	☐	☐	☐		

The 'Advanced' tab is also visible, showing the 'CREATE TABLE' SQL statement:

```
1 CREATE TABLE "Code" (  
2   "ID" INTEGER,  
3   "FileName" TEXT,  
4   "Description" TEXT,  
5   "PythonCode" TEXT,  
6   "Args" TEXT,  
7   PRIMARY KEY ("ID" AUTOINCREMENT)  
8 );
```

On the right side of the application, the 'SQL Log' panel is visible, showing the executed SQL statement:

```
1 PRAGMA foreign_keys = '1';  
2 PRAGMA database_list;  
3 SELECT type,name,sql,tbl_name FROM "main".sqlite_master;  
4 PRAGMA encoding;  
5 PRAGMA foreign_keys;  
6 PRAGMA foreign_keys = '0';  
7 PRAGMA collation_list;  
8 SAVEPOINT "db4s_editttable_17622660635142464";  
9
```

- <https://sqlitebrowser.org/dl/>

Select all

0 selected

The screenshot shows the phpMyAdmin interface for a server named 'localhost'. The 'Databases' tab is selected. On the left, a sidebar lists several databases: 'chart', 'cmm', 'db', 'demo', 'dynamic_textbox', 'e-quote', 'ecommerce', and 'editor'. The main content area displays the 'Databases' section, which includes a 'Create database' form with a 'Database name' field and a 'Create' button. Below this is a 'Filters' section with a 'Containing the word:' input field. At the bottom, a table lists the existing databases:

Database	Collation	Action
☐ chart	utf8mb4_general_ci	Check privileges
☐ cmm	utf8mb4_general_ci	Check privileges

3. Các thành phần cơ bản của cơ sở dữ liệu

- Bảng (table): lưu trữ dữ liệu theo dạng hàng/cột
- Khóa chính (Primary Key), khóa ngoại (Foreign Key)
- Quan hệ giữa các bảng
- Các truy vấn cơ bản

- Cách ứng dụng CSDL giúp lưu trữ và xử lý dữ liệu hiệu quả

5. Kiến thức mở rộng

- Mô hình dữ liệu quan hệ (RDBMS)
- Mô hình ERD (Entity-Relationship Diagram)

5.2 Ngôn ngữ hỏi SQL (2 tiết) Ôn lại môn Hệ quản trị CSDL

Mục tiêu

- Hiểu và sử dụng thành thạo câu lệnh SQL cơ bản để thao tác với cơ sở dữ liệu
- Biết cách thực hiện truy vấn, chèn, sửa, xóa dữ liệu trong SQL

Nội dung chính

1. Giới thiệu SQL (Structured Query Language)

- SQL là ngôn ngữ tiêu chuẩn để tương tác với các hệ quản trị cơ sở dữ liệu quan hệ.

2. Các lệnh SQL cơ bản

- **SELECT**: truy vấn dữ liệu
- **INSERT INTO**: chèn dữ liệu mới
- **UPDATE**: cập nhật dữ liệu
- **DELETE**: xóa dữ liệu

- **DROP TABLE:** xóa bảng

3. Điều kiện lọc dữ liệu với WHERE

- Sử dụng các toán tử so sánh, logic (AND, OR, NOT)

4. Sắp xếp dữ liệu với ORDER BY

- Sắp xếp theo tăng dần (ASC) hoặc giảm dần (DESC)

5. Hàm tổng hợp

- COUNT, SUM, AVG, MAX, MIN

6. Các câu truy vấn nâng cao

- JOIN (INNER, LEFT, RIGHT)
- GROUP BY, HAVING

5.3 Giới thiệu về ADO.NET (6 tiết)

Mục tiêu

- Hiểu kiến trúc và các thành phần chính của ADO.NET
- Biết cách sử dụng ADO.NET để kết nối và thao tác với cơ sở dữ liệu trong ứng dụng C#
- Viết chương trình thực tế tương tác với CSDL bằng ADO.NET

Nội dung chính

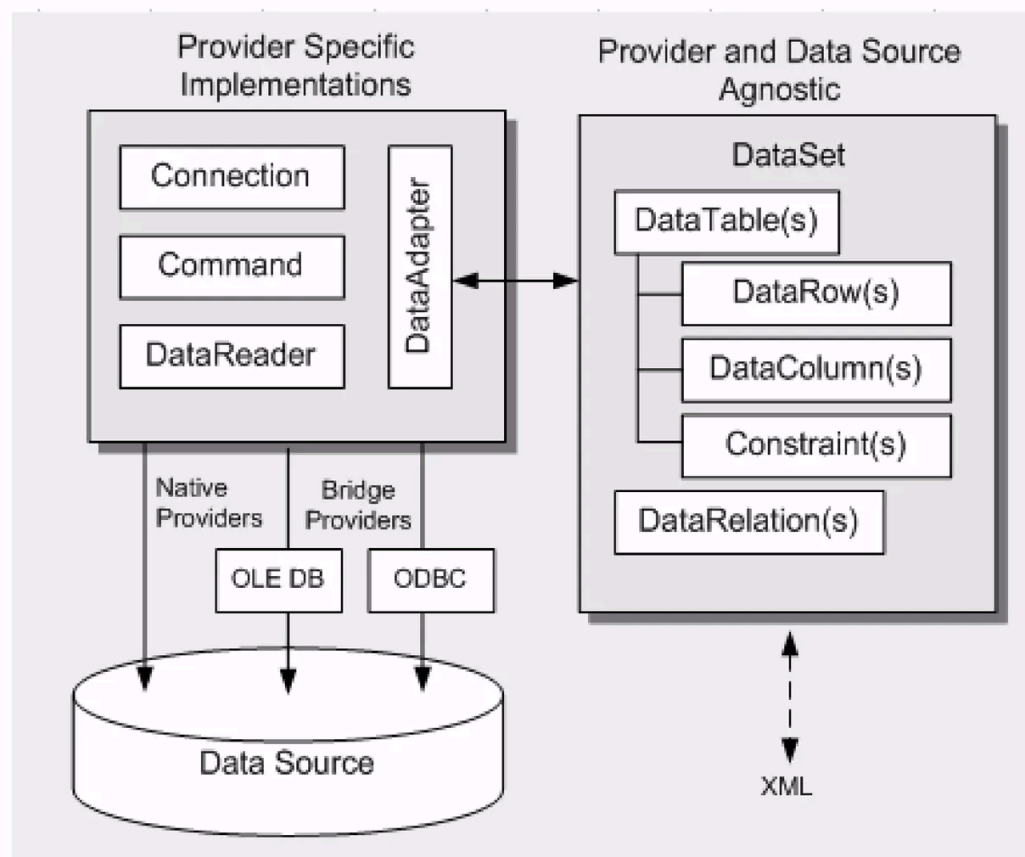
1. Giới thiệu ADO.NET

Select all

0 selected

2. Kiến trúc ADO.NET

- **Connection:** quản lý kết nối đến CSDL (ví dụ: SqlConnection)
- **Command:** thực thi câu lệnh SQL hoặc thủ tục lưu (ví dụ: SqlCommand)
- **DataReader:** đọc dữ liệu theo kiểu forward-only, read-only
- **DataSet:** lưu trữ dữ liệu trong bộ nhớ, có thể chỉnh sửa, cập nhật
- **DataAdapter:** kết nối giữa DataSet và CSDL để cập nhật dữ liệu
- **Transaction:** quản lý các thao tác giao dịch dữ liệu
- **Parameters:** sử dụng để bảo vệ khỏi tấn công SQL Injection



3. Kết nối với cơ sở dữ liệu

- Chuỗi kết nối (connection string)

Select all

0 selected

```
using System.Data.SqlClient;

class Program
{
    static void Main()
    {
        // 📦 Chuỗi kết nối (Connection String)
        string connectionString = "Server=localhost;Database=MyDatabase;User
Id=myUsername;Password=myPassword;";

        // Tạo đối tượng kết nối
        using (SqlConnection connection = new SqlConnection(connectionString))
        {
            try
            {
                // 🌀 Mở kết nối
                connection.Open();
                Console.WriteLine("Kết nối thành công!");

                // Thực hiện một truy vấn thử
                string query = "SELECT COUNT(*) FROM Users";
                SqlCommand command = new SqlCommand(query, connection);
                int count = (int)command.ExecuteScalar();
                Console.WriteLine($"Số lượng người dùng: {count}");
            }
            catch (Exception ex)
            {
                Console.WriteLine("Lỗi: " + ex.Message);
            }
        }
    }
}
```



```
        if (connection.State == System.Data.ConnectionState.Open)
        {
            connection.Close();
            Console.WriteLine("Đã đóng kết nối.");
        }
    }
}
```

4. Thực thi câu lệnh SQL

- Thực thi câu lệnh SELECT bằng SqlDataReader
- Thực thi câu lệnh INSERT, UPDATE, DELETE bằng ExecuteNonQuery()
- Thực thi thủ tục lưu

5. Sử dụng DataSet và DataAdapter

- Đọc dữ liệu vào DataSet
- Cập nhật dữ liệu từ DataSet về cơ sở dữ liệu
- Phân biệt DataReader và DataSet

6. Xử lý lỗi và quản lý tài nguyên

- Sử dụng `try-catch-finally`
- Sử dụng `using` để tự động giải phóng tài nguyên

Select all

0 selected

- Thực hiện CRUD (Create, Read, Update, Delete) dữ liệu
- Hiển thị dữ liệu ra giao diện đơn giản



Bài tập và dự án đề xuất:

1. Bài tập SQL:

- Viết câu lệnh SQL để tạo bảng, chèn dữ liệu, truy vấn dữ liệu đơn giản.
- Thực hiện các truy vấn có điều kiện, sắp xếp và sử dụng hàm tổng hợp.

2. Bài tập ADO.NET:

- Viết chương trình C# kết nối đến SQL Server/MySQL bằng ADO.NET.
- Thực hiện các thao tác CRUD trên bảng dữ liệu.
- Xử lý ngoại lệ khi thao tác với cơ sở dữ liệu.

3. Dự án nhỏ:

- Xây dựng ứng dụng quản lý sinh viên đơn giản sử dụng C# và SQL Server.
- Giao diện nhập liệu, hiển thị danh sách sinh viên, tìm kiếm và cập nhật dữ liệu.



(Không có khối lượng trong đề cương)

Chương 1: Giới thiệu về .NET Core (6 tiết)

Chuẩn đầu ra: CĐR 1, 2

**BÀI HƯỚNG DẪN
CÁC BƯỚC TRIỂN KHAI
MỘT ỨNG DỤNG. MỞ MÃ.
VÍ DỤ APP THI HÀNH CODE PYTHON**

Bài 1: Làm quen với Editor MS Visual Studio (2 tiết)

- Mục tiêu:
 - Làm quen với môi trường phát triển Visual Studio
 - Tạo project mới, cấu trúc dự án, chạy ứng dụng console đơn giản

Select all

0 selected

- Viết chương trình "Hello World"
- Thực hành biên dịch, chạy và debug cơ bản

Bài 2: Thực hành sử dụng biến, hằng, toán tử (4 tiết)

- Mục tiêu:
 - Hiểu và sử dụng biến, hằng số trong C#
 - Sử dụng các toán tử số học, logic, so sánh
- Nội dung thực hành:
 - Khai báo biến, hằng
 - Thực hiện các phép toán và xuất kết quả
 - Viết chương trình tính toán đơn giản
 - Thực hành xử lý lỗi cơ bản khi nhập dữ liệu

Chương 2: Giao diện người dùng (User Interface) (6 tiết)

Chuẩn đầu ra: CĐR 3

Bài 3: Ứng dụng Windows Form đầu tiên (2 tiết)

- Mục tiêu:
 - Tạo và chạy ứng dụng Windows Form đơn giản

Select all

0 selected

- Tạo Form, thêm các điều khiển cơ bản (Button, Label, TextBox)
- Xử lý sự kiện nhấn nút
- Hiển thị thông báo

Bài 4: Xử lý chuỗi (4 tiết)

- Mục tiêu:
 - Thao tác với chuỗi trong C#
 - Sử dụng các phương thức xử lý chuỗi
- Nội dung thực hành:
 - Nhập xuất chuỗi trên form
 - Các thao tác nối, cắt, tìm kiếm chuỗi
 - Viết chương trình xử lý chuỗi đơn giản qua giao diện Windows Form

Chương 3: Các kiểu dữ liệu & Bẫy lỗi trong C# (6 tiết)

Chuẩn đầu ra: CĐR 2, 4

Bài 5: Thực hành các cấu trúc điều khiển (2 tiết)

- Mục tiêu:
 - Sử dụng các cấu trúc điều kiện và vòng lặp trong C#
- Nội dung thực hành:

Select all

0 selected

- Giải quyết các bài toán đơn giản

Bài 6: Thiết kế MDI form và Menu (4 tiết)

- Mục tiêu:
 - Tạo ứng dụng nhiều cửa sổ con (MDI)
 - Thiết kế và quản lý Menu trong Windows Form
- Nội dung thực hành:
 - Tạo MDI Parent và Child Forms
 - Thêm MenuStrip, ToolStrip và xử lý sự kiện menu
 - Thực hành chuyển đổi giữa các cửa sổ

Chương 4: Lập Trình Hướng Đối Tượng với C# (6 tiết)

Chuẩn đầu ra: CĐR 4

Bài 7: Thực hành viết Lớp (Class) trong C# (2 tiết)

- Mục tiêu:
 - Hiểu cách định nghĩa và sử dụng class, thuộc tính, phương thức
- Nội dung thực hành:
 - Viết lớp với thuộc tính, phương thức
 - Tạo đối tượng, gọi phương thức

Select all

0 selected

- Mục tiêu:
 - Hiểu và áp dụng kế thừa, đa hình
- Nội dung thực hành:
 - Viết lớp kế thừa
 - Ghi đè phương thức (override)
 - Viết chương trình demo đa hình

Bài 9: Sử dụng các đối tượng danh sách trong lập trình Windows Form (2 tiết)

- Mục tiêu:
 - Sử dụng các collection như List, Dictionary
 - Hiển thị dữ liệu lên các điều khiển như ListBox, ComboBox, DataGridView
- Nội dung thực hành:
 - Quản lý danh sách sinh viên, sản phẩm
 - Thêm, sửa, xóa phần tử trong danh sách
 - Cập nhật giao diện theo danh sách

Chương 5: Cơ sở dữ liệu & C# (6 tiết)

Chuẩn đầu ra: CĐR 4

Bài 10: Các thao tác cơ bản trên SQL Server (2 tiết)

- Nội dung thực hành:
 - Tạo bảng sinh viên, bảng sản phẩm
 - Viết câu lệnh INSERT, UPDATE, DELETE
 - Thực hiện truy vấn SELECT

Bài 11: Đọc dữ liệu từ SQL Server lên form (2 tiết)

- Mục tiêu:
 - Kết nối C# với SQL Server bằng ADO.NET
 - Hiển thị dữ liệu trên Windows Form
- Nội dung thực hành:
 - Viết code sử dụng SqlConnection, SqlCommand, SqlDataAdapter
 - Hiển thị dữ liệu lên DataGridView
 - Thực hiện truy vấn đơn giản

Bài 12: Làm quen với các thao tác trên Database (1 tiết)

- Mục tiêu:
 - Thực hiện các thao tác CRUD thông qua giao diện
- Nội dung thực hành:
 - Thêm, sửa, xóa dữ liệu qua form

Select all

0 selected

- Mục tiêu:
 - Hiểu và thực hành tạo báo cáo với Crystal Report trong C#
- Nội dung thực hành:
 - Tạo báo cáo đơn giản
 - Kết nối báo cáo với dữ liệu SQL Server
 - Xuất báo cáo

Tổng kết chương 6

- Qua 30 tiết thực hành, sinh viên sẽ có cơ hội tiếp xúc thực tế với .NET Core, C#, giao diện người dùng, lập trình hướng đối tượng và kết nối cơ sở dữ liệu.
- Các bài tập được thiết kế tăng dần về độ khó và bám sát chuẩn đầu ra, giúp sinh viên củng cố kỹ năng lập trình và phát triển ứng dụng thực tế.
- Các kiến thức thực hành này cũng là nền tảng quan trọng để phát triển các ứng dụng phức tạp hơn trong tương lai.



Hướng dẫn làm bài tập lớn 



Select all

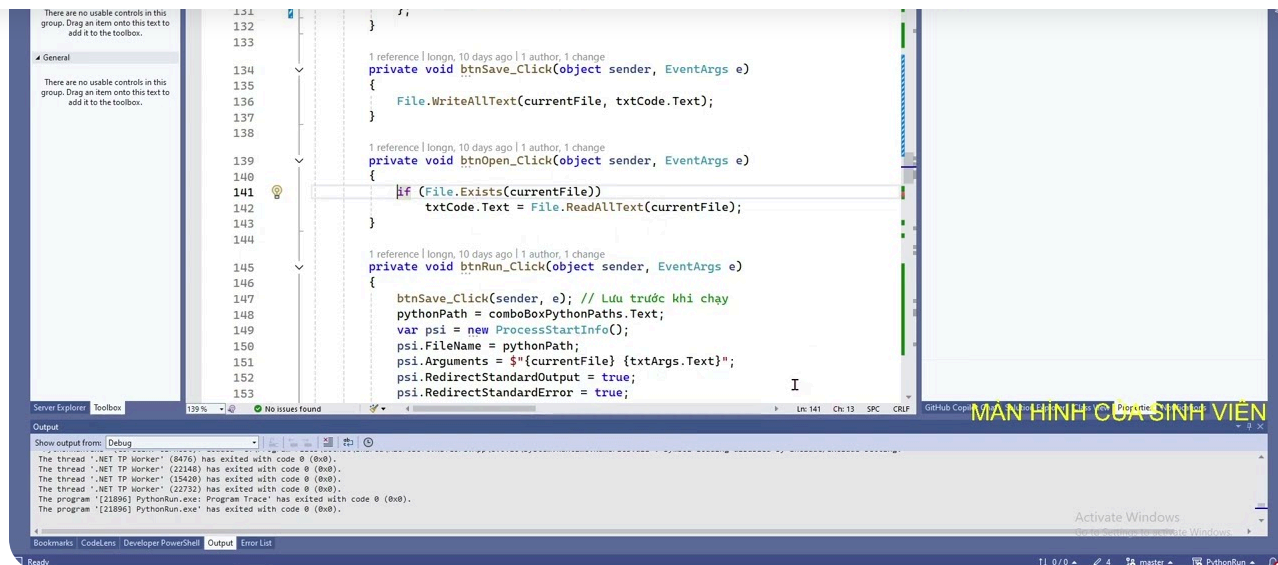
0 selected

**HƯỚNG DẪN LÀM BÀI TẬP LỚN
MÔN HỌC: WINDOWS FORMS
PROGRAMMING IN .NET**

**BÀI HƯỚNG DẪN
CÁC BƯỚC TRIỂN KHAI
MỘT ỨNG DỤNG. MỞ MÃ.
VÍ DỤ APP THI HÀNH CODE PYTHON**

Select all

0 selected



Hình thức và nội dung trình bày của báo cáo:

MẪU CẤU TRÚC BÁO CÁO BÀI TẬP LỚN

Trang bìa

Trường / Khoa / Bộ môn

BÁO CÁO BÀI TẬP LỚN

Tên môn học: Lập trình C# WinForms

Tên đề tài: (VD: Phần mềm Quản Lý Sinh viên WinForms)

Sinh viên thực hiện: Nguyễn Văn A

Mã số sinh viên: 22IT123

Lớp: CNTT-K22

Mục lục

(Tự động tạo bằng MS Word hoặc soạn thủ công – ghi rõ số trang cho từng phần)

1. Giới thiệu đề tài

1.1. Lý do chọn đề tài

- Nêu lý do thực tế, tầm quan trọng, ý nghĩa của ứng dụng.
- Ví dụ: “Quản lý sinh viên thủ công tốn thời gian, dễ sai sót. Do đó cần phần mềm hỗ trợ quản lý tự động.”

1.2. Mục tiêu của đề tài

- Trình bày mục tiêu cụ thể, rõ ràng:
 - Xây dựng phần mềm WinForms có thể thêm, sửa, xóa, tìm kiếm dữ liệu.
 - Lưu trữ thông tin vào cơ sở dữ liệu SQL Server.
 - Cung cấp giao diện thân thiện và báo cáo tổng hợp.

1.3. Phạm vi thực hiện

- Nêu rõ phạm vi: chức năng nào được thực hiện, chức năng nào chưa.
- Ví dụ: Chỉ tập trung vào quản lý sinh viên – chưa bao gồm điểm hoặc học phí.

2. Phân tích yêu cầu

2.1. Mô tả bài toán

- Mô tả nghiệp vụ: người dùng là ai, cần thao tác gì.

Select all

0 selected

- Quản lý người dùng (đăng nhập, đăng xuất).
- Quản lý danh sách sinh viên.
- Quản lý lớp học.
- Tìm kiếm – thống kê.

2.3. Yêu cầu phi chức năng

- Giao diện thân thiện.
- Tốc độ xử lý nhanh.
- Dữ liệu được lưu trữ an toàn.

3. Thiết kế hệ thống

3.1. Thiết kế cơ sở dữ liệu

- Bảng dữ liệu chính (tên bảng, khóa chính, khóa ngoại).
- Mô tả các bảng:

Tên bảng	Mô tả	Các cột dữ liệu
SINHVIEN	Lưu thông tin sinh viên	MaSV (PK), HoTen, Lop, NgaySinh, GioiTinh
LOP	Lưu thông tin lớp	MaLop (PK), TenLop, KhoaHoc

3.2. Thiết kế giao diện (UI/UX)

- Mô tả hoặc chụp hình các form:
 - Form đăng nhập

4. Cài đặt chương trình

4.1. Môi trường phát triển

- Ngôn ngữ: C# WinForms
- IDE: Visual Studio 2022
- CSDL: SQL Server 2019
- Framework: .NET 6 (hoặc .NET Framework 4.8)

4.2. Cấu trúc thư mục dự án

/ProjectName

```
|— /DAL      (Data Access Layer)
|— /BLL      (Business Logic Layer)
|— /GUI      (WinForms giao diện)
|— /Models   (Lớp dữ liệu)
|— /Reports  (File báo cáo RDLC)
|— App.config
```

4.3. Các chức năng chính

Mỗi chức năng trình bày gồm:

- Mô tả mục đích
- Giao diện (ảnh chụp)
- Mã nguồn chính (trích đoạn quan trọng, có chú thích)

Select all

0 selected

```
{  
    Student s = new Student()  
    {  
        MaSV = txtMaSV.Text,  
        HoTen = txtHoTen.Text,  
        Lop = cbLop.SelectedValue.ToString()  
    };  
    studentService.AddStudent(s);  
    LoadData();  
}
```



5. Kết quả thực nghiệm

5.1. Giao diện chính

(Chèn hình giao diện chạy thực tế, có chú thích)

5.2. Các chức năng hoạt động

- Mô tả và minh họa bằng ảnh chụp: thêm, xóa, tìm kiếm, in báo cáo...

5.3. Kết quả báo cáo / thống kê

- Ví dụ: Biểu đồ thống kê sinh viên theo lớp, theo giới tính,...



6. Đánh giá – Hạn chế – Hướng phát triển

6.1. Đánh giá

- Hệ thống hoạt động ổn định, đáp ứng đúng yêu cầu bài toán.
- Giao diện thân thiện, dễ sử dụng.

- Chưa triển khai mạng LAN hoặc API.

6.3. Hướng phát triển

- Nâng cấp lên mô hình 3 lớp hoàn chỉnh.
- Xây dựng phiên bản web (ASP.NET).
- Kết nối cơ sở dữ liệu online (Azure, Firebase).

7. Kết luận

- Tóm tắt lại mục tiêu, kết quả đạt được.
- Nêu cảm nhận, bài học kinh nghiệm khi làm bài tập lớn.

8. Tài liệu tham khảo

- Giáo trình Lập trình C# – NXB Giáo dục.
- Microsoft Documentation: <https://learn.microsoft.com/dotnet>
- Các video hướng dẫn, nguồn tham khảo (nếu có).

9. Phụ lục (nếu cần)

- Mã nguồn quan trọng (Appendix).
- Hình ảnh demo chi tiết.
- Cấu trúc CSDL dạng script **.sql**.

Select all

0 selected

- Giãn dòng: 1.5.
- Lề: Trên 2.5 cm, Dưới 2.5 cm, Trái 3.0 cm, Phải 2.0 cm.
- Đánh số trang, canh giữa tiêu đề, in rõ tên đề tài.

BÀI GIẢNG NET 1 LẬP TRÌNH C# WINFORM - BÀI TẬP LỚN

Danh sách các bài tập có thể lựa chọn để thực hiện:

10 bài đơn giản.

1. Quản lý Danh bạ Cá nhân

Select all

0 selected

• Giao diện nhập hệ tên, số điện thoại, email.

- Nút **Thêm, Xóa, Sửa** liên hệ.
- Hiển thị danh sách liên hệ trong **ListBox** hoặc **DataGridView**.
- Lưu dữ liệu tạm trong danh sách (List<Contact>).

2. Quản lý Sinh viên

Mục tiêu: Làm việc với **DataGridView** và xử lý CRUD cơ bản.

Yêu cầu:

- Nhập: Mã SV, Họ tên, Ngày sinh, Giới tính, Lớp.
- Chức năng: Thêm – Xóa – Sửa – Tìm kiếm.
- Hiển thị danh sách sinh viên trong **DataGridView**.
- Kiểm tra hợp lệ dữ liệu (vd: không để trống mã SV).

3. Quản lý Thư viện

Mục tiêu: Làm việc với dữ liệu nhiều bảng và ràng buộc quan hệ.

Yêu cầu:

- Quản lý **Sách, Độc giả, Mượn – Trả sách**.
- Giao diện có nhiều tab hoặc form con.
- Xử lý logic khi mượn/trả (ví dụ: không cho mượn nếu quá 3 quyển).
- Lưu dữ liệu vào file **.txt** hoặc **.json**.

- Quản lý sản phẩm: mã, tên, đơn giá, số lượng.
- Form bán hàng: chọn sản phẩm, nhập số lượng → tính tiền.
- Tự động tính tổng tiền, thuế VAT, tiền trả lại khách.
- Có thể lưu hóa đơn ra file `.txt` hoặc `.csv`.

5. Ứng dụng Quản lý Điểm Sinh viên

Mục tiêu: Kết hợp tính toán, xử lý dữ liệu và hiển thị.

Yêu cầu:

- Nhập điểm các môn học (toán, lý, hóa).
- Tính điểm trung bình, xếp loại học lực.
- Hiển thị danh sách sinh viên và kết quả trong `DataGridView`.
- Thêm chức năng lọc theo học lực ("Giỏi", "Khá", "Trung bình").

6. Quản lý Khách sạn (Đặt phòng)

Mục tiêu: Quản lý dữ liệu có trạng thái và điều kiện.

Yêu cầu:

- Quản lý phòng (mã phòng, loại phòng, tình trạng).
- Quản lý khách hàng, đặt phòng, trả phòng.
- Khi phòng đã đặt, không cho đặt trùng.

Mục tiêu: Kết nối dữ liệu với SQL Server.

Yêu cầu:

- CSDL **NhanVien** có các trường: MaNV, TenNV, NgaySinh, ChucVu, Luong.
- Giao diện WinForms kết nối CSDL qua ADO.NET.
- Chức năng: Thêm, Xóa, Sửa, Tìm kiếm nhân viên.
- Hiển thị dữ liệu bằng **DataGridView** và tự động cập nhật.

8. Quản lý Hóa đơn Bán hàng (CSDL + Giao diện)

Mục tiêu: Rèn kỹ năng thao tác CSDL nhiều bảng (Quan hệ 1-n).

Yêu cầu:

- Bảng **HoaDon** và **ChiTietHoaDon**.
- Giao diện nhập hóa đơn → thêm sản phẩm → tính tổng tiền.
- Chức năng in hoặc xuất hóa đơn ra file **.pdf** (sử dụng thư viện iTextSharp).

9. Ứng dụng Quản lý Người dùng (Login/Register)

Mục tiêu: Làm quen với xác thực và bảo mật cơ bản.

Yêu cầu:

- Giao diện **Đăng nhập – Đăng ký – Đổi mật khẩu**.
- Lưu thông tin người dùng vào CSDL hoặc file **.json**.
- Mã hóa mật khẩu khi lưu (MD5 hoặc SHA256).

Select all

0 selected

Mục tiêu: Ứng dụng xử lý dữ liệu, biểu đồ và báo cáo.

Yêu cầu:

- Nhập dữ liệu doanh thu theo ngày/tháng/năm.
- Tính tổng, trung bình, so sánh các tháng.
- Hiển thị bằng **Chart Control** (biểu đồ cột, đường).
- Xuất báo cáo doanh thu ra file **.pdf** hoặc **.excel**.

20 bài cho sinh viên khá:

20 BÀI TẬP THỰC HÀNH C# WINFORMS NÂNG CAO

I. Nhóm Ứng dụng Quản lý (CRUD + SQL Server + UI nâng cao)

1. Quản lý Nhà sách

- Quản lý **sách, tác giả, thể loại, hóa đơn bán hàng**.
- Chức năng: tìm kiếm theo tên, thể loại, tác giả.
- Tính năng nâng cao: gợi ý tự động (AutoComplete), xuất hóa đơn ra PDF.

2. Quản lý Bệnh viện – Phòng khám

- Quản lý bệnh nhân, bác sĩ, lịch khám, toa thuốc.
- Kết hợp **DateTimePicker**, **ComboBox liên kết động** (chọn bác sĩ → tự lọc lịch trống).
- Thêm tính năng: cảnh báo trùng lịch khám.

Select all

0 selected

- Tính điểm thưởng khách hàng.
- Báo cáo doanh thu theo tháng, biểu đồ Chart.
- Sử dụng **SQL Server** + **Stored Procedure**.

4. Quản lý Nhà hàng – Đặt bàn

- Form sơ đồ bàn ăn bằng **Panel + Button** động.
- Tạo hóa đơn tự động khi khách ngồi bàn.
- Quản lý thực đơn, nhân viên phục vụ, doanh thu trong ngày.

5. Quản lý Kho hàng

- Quản lý nhập – xuất – tồn.
- Tự động cập nhật số lượng khi có giao dịch.
- Báo cáo hàng tồn thấp, hàng sắp hết hạn.
- Dùng **DataGridView** với tính năng lọc và sắp xếp động.

6. Quản lý Học phí Sinh viên

- Quản lý sinh viên, lớp, học phí, công nợ.
- Tính năng: thống kê nợ học phí, in biên lai.
- Kết nối SQL + xuất báo cáo **.pdf**.

7. Quản lý Tuyển dụng

- Tích hợp tính năng upload CV (file PDF/Docx).

8. Quản lý Kho sách Thư viện (phiên bản mở rộng)

- Dùng **Entity Framework** thay vì ADO.NET.
- Quản lý mượn – trả, hạn mượn, tính tiền phạt tự động.
- Có **Login phân quyền (Admin / Thủ thư / Độc giả)**.

9. Quản lý Cửa hàng Thời trang

- Giao diện có hình ảnh sản phẩm.
- Thêm sản phẩm bằng cách **chọn ảnh từ file**.
- Báo cáo doanh thu theo sản phẩm, tháng, nhân viên bán hàng.

10. Quản lý Trung tâm Ngoại ngữ

- Quản lý học viên, lớp học, giáo viên, lịch học.
- Chức năng: tự động phân lớp theo trình độ đầu vào.
- Thống kê tỉ lệ lên lớp, học viên hoàn thành khóa học.

II. Nhóm Ứng dụng Xử lý Dữ liệu – Giao diện – Tích hợp Công nghệ

11. Quản lý Tài chính Cá nhân

- Ghi chép thu chi, thống kê chi tiêu theo loại (ăn uống, học tập, di chuyển...).
- Hiển thị biểu đồ Pie Chart hoặc Line Chart.

Select all

0 selected

- Tạo, sửa, xóa lịch hẹn cá nhân.
- Dùng **MonthCalendar** và **NotifyIcon** để nhắc lịch.
- Gợi ý thêm: kết hợp **SQLite** để lưu trữ dữ liệu cục bộ.

13. Ứng dụng Chat LAN (WinForms Socket)

- Kết nối giữa 2 máy tính trong cùng mạng LAN.
- Cho phép gửi tin nhắn, file nhỏ.
- Dùng **System.Net.Sockets** để lập trình mạng cơ bản.

14. Ứng dụng Quản lý Email giả lập

- Giao diện giống hộp thư điện tử: Hộp đến, Hộp đi, Soạn thư.
- Lưu trữ dữ liệu vào SQLite.
- Mô phỏng gửi thư nội bộ (User A → User B).

15. Ứng dụng Quản lý Ảnh – Album

- Cho phép thêm ảnh, nhóm theo album.
- Hiển thị ảnh bằng **FlowLayoutPanel** và **PictureBox**.
- Chức năng: Xem slideshow, xóa ảnh, đổi tên album.

16. Quản lý Hệ thống Chấm công Nhân viên

- Chấm công bằng form nhập mã nhân viên và ngày giờ.

17. Quản lý Vé máy bay / Xe khách

- Quản lý tuyến, chuyến, khách hàng, vé.
- Khi chọn tuyến → tự động hiển thị các chuyến tương ứng.
- Xử lý logic “đặt vé – hủy vé – tính tiền”.

18. Ứng dụng Quản lý Bảo hành Sản phẩm

- Quản lý thông tin khách hàng, sản phẩm, thời hạn bảo hành.
- Cảnh báo khi sản phẩm sắp hết hạn bảo hành.
- Giao diện có phân quyền nhân viên và quản lý.

19. Quản lý Phòng trọ / Chung cư mini

- Quản lý người thuê, hợp đồng, chỉ số điện nước.
- Tự động tính tiền phòng mỗi tháng.
- In hóa đơn và báo cáo chi tiết.

20. Quản lý Vận chuyển – Giao hàng

- Quản lý đơn hàng, tài xế, lộ trình, trạng thái giao hàng.
- Theo dõi đơn theo tình trạng: “Đang giao / Hoàn thành / Hủy”.
- Biểu đồ thống kê số đơn theo tài xế / khu vực.

Select all

0 selected

Kỹ năng	Mô tả
Giao diện WinForms	Thiết kế form chuyên nghiệp, sử dụng control nâng cao (TabControl, Panel động, Chart, DataGridView custom).
Xử lý dữ liệu	Thành thạo CRUD, LINQ, Entity Framework, SQL Server, SQLite.
Xử lý nghiệp vụ	Áp dụng mô hình 3 lớp (GUI – BLL – DAL).
Bảo mật & đăng nhập	Quản lý người dùng, phân quyền, mã hóa dữ liệu.
Báo cáo & thống kê	Biểu đồ, báo cáo PDF/Excel, tổng hợp dữ liệu động.
Tư duy phần mềm	Biết thiết kế module, chia lớp, tách xử lý logic – giao diện.



Gợi ý hướng phát triển tiếp theo:

Sau khi hoàn thành 30 bài này, sinh viên có thể:

1. **Làm đồ án cuối kỳ / tốt nghiệp:** Ví dụ “Phần mềm Quản lý Nhà hàng / Khách sạn / Cửa hàng tiện lợi”.
2. **Chuyển sang học WPF hoặc ASP.NET** để mở rộng lên web/app.
3. **Kết hợp AI hoặc Machine Learning cơ bản** cho phần gợi ý sản phẩm hoặc dự báo doanh thu.

🌟 20 BÀI TẬP WINFORMS NÂNG CAO DÀNH CHO SINH VIÊN XUẤT SẮC

🌿 I. NHÓM ỨNG DỤNG QUẢN LÝ PHỨC HỢP – MÔ HÌNH 3 LỚP + ENTITY FRAMEWORK + REPORT

1. Phần mềm Quản lý Doanh nghiệp mini (ERP cơ bản)

- Sử dụng **Entity Framework Core**, mô hình **Repository Pattern**.
- Báo cáo đa tầng, dashboard thống kê.

2. Quản lý Khách sạn thông minh

- Quản lý đặt phòng, check-in/out, tính tiền dịch vụ tự động.
- Có module quản lý nhân viên, khách hàng thân thiết.
- Tích hợp API gửi SMS/email khi đặt phòng thành công.
- UI dạng dashboard tổng hợp tình trạng phòng.

3. Quản lý Cửa hàng xe máy / ô tô

- Quản lý sản phẩm, khách hàng, hợp đồng trả góp, bảo hành.
- Tính toán tài chính trả góp, tự sinh mã hợp đồng.
- Báo cáo doanh thu – lợi nhuận bằng biểu đồ động.

4. Quản lý Trường học toàn diện (School Manager)

- Quản lý học sinh, giáo viên, điểm, thời khóa biểu, lớp học.
- Tích hợp **tự động tính điểm trung bình, xếp loại, xuất bảng điểm Excel**.
- Dùng **Crystal Reports / RDLC Report** để in phiếu điểm.

5. Quản lý Bán vé Rạp phim

- Quản lý lịch chiếu, phim, ghế, vé, khách hàng.

6. Quản lý Phòng tập Gym

- Quản lý gói tập, huấn luyện viên, học viên, thời gian tập.
- Theo dõi tiến độ luyện tập, cân nặng, chỉ số sức khỏe.
- Báo cáo thống kê bằng biểu đồ.

7. Quản lý Khóa học Online (LMS mini)

- Module: khóa học, học viên, bài học, kết quả kiểm tra.
- Cho phép upload video (giả lập link nội bộ).
- Theo dõi tiến độ học, điểm quiz.

8. Quản lý Bảo trì thiết bị công nghiệp

- Quản lý thiết bị, kế hoạch bảo trì, nhân viên kỹ thuật.
- Gửi thông báo thiết bị cần bảo trì định kỳ (NotifyIcon / Email API).
- Có form nhập nhật ký bảo trì.

9. Quản lý Siêu thị Online (phần mềm bán hàng POS)

- Giao diện bán hàng nhanh bằng **Barcode Scanner** hoặc nhập mã SP.
- Tích hợp in hóa đơn, thanh toán tiền mặt / chuyển khoản.
- Sử dụng **EF + LINQ + DataGridView custom**.

10. Hệ thống Quản lý Bệnh viện (Hospital Manager)

- Báo cáo số bệnh nhân, thống kê thuốc sử dụng.

II. NHÓM ỨNG DỤNG CÔNG NGHỆ – ĐA LUỒNG, XỬ LÝ FILE, API, BẢO MẬT

11. Ứng dụng Backup & Restore Database WinForms

- Giao diện chọn file **.bak** và phục hồi SQL Server.
- Có tính năng lên lịch **tự động backup** (Timer + Task Scheduler).
- Cho phép **mã hóa file backup** bằng AES.

12. Ứng dụng Quản lý File (File Explorer mini)

- Mô phỏng Windows Explorer: xem, xóa, đổi tên, sao chép file.
- Sử dụng **TreeView + ListView + ContextMenuStrip**.
- Thêm chức năng **tim file nhanh bằng đa luồng (Threading)**.

13. Trình tải File từ Internet (Download Manager)

- Nhập URL → tải file về máy (sử dụng **HttpClient**).
- Hiển thị tiến trình tải bằng ProgressBar.
- Có **resume download** khi bị ngắt kết nối.

14. Ứng dụng Mã hóa – Giải mã File

- Chọn file → mã hóa AES / RSA → lưu ra file khác.
- Có giao diện trực quan với thanh tiến trình.

Select all

0 selected

- Dùng `TcpListener` và `TcpClient`.
- Quản lý nhiều người dùng cùng lúc (Socket đa luồng).
- Cho phép gửi tin, file, và emoji cơ bản.

16. Ứng dụng Quản lý API (REST Client)

- Giao diện test API (GET, POST, PUT, DELETE).
- Cho phép hiển thị JSON và convert sang DataGridView.
- Có bộ nhớ lưu các request (history).

17. Hệ thống Đăng nhập với Bảo mật JWT

- WinForms + API Web (ASP.NET Core).
- Giao diện đăng nhập gọi API nhận token JWT.
- Kiểm tra token trước khi truy cập tính năng chính.

18. Phần mềm Quản lý Mạng nội bộ

- Quét thiết bị trong LAN (IP, tên máy, trạng thái online).
- Gửi thông báo (ping, message LAN).
- Hiển thị bằng ListView có icon trạng thái.

19. Ứng dụng Quản lý Hóa đơn Điện tử (E-Invoice)

- Nhập hóa đơn, khách hàng, sản phẩm.

20. Phần mềm Quản lý Trí tuệ nhân tạo (AI mini App)

- Giao diện nhập câu hỏi → gọi API ChatGPT/OpenAI.
- Lưu lịch sử hội thoại, cho phép export text.
- Có chế độ “tư vấn học tập” hoặc “phân tích dữ liệu đầu vào”.

KỸ NĂNG SINH VIÊN XUẤT SẮC PHÁT TRIỂN SAU 60 BÀI (tổng hợp 10 + 20 + 20):

Nhóm kỹ năng	Mục tiêu đạt được
 Quản lý dữ liệu nâng cao	Entity Framework Core, Stored Procedure, LINQ, Transaction, EF Migration.
 Kiến trúc phần mềm	MVC, Repository Pattern, Layered Architecture (3-tier, n-tier).
 Bảo mật	Hash mật khẩu, mã hóa dữ liệu AES/RSA, xác thực JWT.
 Tích hợp API	Làm việc với RESTful API, đọc JSON, xử lý dữ liệu từ server.
 Xử lý hệ thống	Threading, Timer, File I/O, System.Diagnostics, Backup/Restore.
 Báo cáo & BI	Sử dụng Chart, RDLC, Excel, PDF, Dashboard tổng hợp.
 Ứng dụng thực tế	Dự án quy mô nhỏ – trung bình tương đương đồ án tốt nghiệp.

Hướng phát triển tiếp theo sau nhóm này:

1. **Chuyển sang WPF hoặc .NET MAUI** → giao diện hiện đại, responsive.
2. **Kết hợp AI / Machine Learning** → dự đoán nhu cầu, nhận diện hình ảnh.
3. **Kết nối đám mây (Cloud)** → lưu dữ liệu trên Firebase, Azure.
4. **Phát triển thành bản thương mại** – tạo installer, update tự động.

Select all

0 selected



Báo cáo bài tập mẫu DOCX  DOCX



Phát triển phần mềm 



Ứng dụng mã mở với .NET

BÀI GIẢNG
MÔN HỌC: MÃ NGUỒN MỞ
ÁP DỤNG MÃ NGUỒN MỞ
PHÁT TRIỂN PHẦN MỀM

Select all

0 selected

<https://github.com/hmy2704/QLNhaSach>



✓ Ôn thi ✎



BỘ CÂU HỎI ÔN TẬP THI VẤN ĐÁP – LẬP TRÌNH .NET 1 (C# WinForms)

Trình bày khoảng 2 mặt A4 Bắt đầu copy từ dòng bên dưới:

NỘI DUNG CHUẨN BỊ THI VẤN ĐÁP HỌC PHẦN NET1 C#

Họ và tên sinh viên:.....

Mã số sinh viên:.....

I. Tổng quan về .NET và C#

1. .NET là gì? Nêu các thành phần chính của nền tảng .NET.
2. So sánh .NET Framework và .NET Core. Ưu điểm của .NET Core?
3. Vai trò của CLR (Common Language Runtime) trong .NET?
4. Ngôn ngữ C# có đặc điểm gì nổi bật? Kể tên 3 tính năng hiện đại của C#.
5. Base Class Library (BCL) là gì? Cho ví dụ một số namespace quan trọng.

II. Thiết kế giao diện người dùng (Windows Forms)

9. Sự khác nhau giữa MenuStrip và ContextMenuStrip? Cho ví dụ sử dụng.

10. TabControl và TreeView dùng để làm gì trong giao diện nâng cao?

III. Kiểu dữ liệu và xử lý lỗi trong C#

11. Phân biệt kiểu giá trị và kiểu tham chiếu trong C#? Cho ví dụ.

12. Nullable type là gì? Cách khai báo và sử dụng?

13. Liệt kê các cấu trúc dữ liệu phổ biến trong C#: Array, List, Dictionary.

14. Enum là gì? Cho ví dụ khai báo và sử dụng.

15. Trình bày cấu trúc try-catch-finally. Nêu 3 loại ngoại lệ thường gặp.

IV. Lập trình hướng đối tượng với C#

16. Property trong C# là gì? So sánh với trường (field).

17. Đa hình (Polymorphism) là gì? Phân biệt overloading và overriding.

18. Thừa kế (Inheritance) trong C#: cú pháp và vai trò.

19. Interface là gì? So sánh Interface với Abstract Class.

20. Cho ví dụ minh họa đa hình với lớp Animal, Dog, Cat.

V. Cơ sở dữ liệu và ADO.NET

21. Cơ sở dữ liệu quan hệ là gì? Nêu vai trò của khóa chính và khóa ngoại.

22. Liệt kê 4 câu lệnh SQL cơ bản (SELECT, INSERT, UPDATE, DELETE) và ý nghĩa.

23. ADO.NET là gì? Các thành phần chính của ADO.NET (Connection, Command, DataReader, DataSet).

24. Phân biệt DataReader và DataSet trong ADO.NET.

25. Trình bày các bước kết nối C# với SQL Server bằng ADO.NET.

Hướng dẫn sử dụng

- In ra **2 mặt A4**, chia thành 2 cột để dễ nhìn.
- Mỗi câu trả lời chuẩn bị ngắn gọn (2–3 dòng hoặc ví dụ code ngắn).
- Tập trung vào **khái niệm, cú pháp, ví dụ thực tế**.

Select all

0 selected

T Add section

“Học cùng AI – Nền tảng học tập cá nhân hóa, hỗ trợ học sinh và giáo viên nâng cao hiệu quả học tập bằng trí tuệ nhân tạo.”

2. Liên hệ

- Email: **support@hoccungai.net**
- Điện thoại: **0123 456 789**
- Địa chỉ: **123 Đường Tri Thức, Hà Nội, Việt Nam**

Info

Trang chủ

Khóa học

Blog

Thảo luận

Hướng dẫn sử dụng

Follow Us

